

COMANDO mkdir NO LINUX

DO BÁSICO AO AVANÇADO

Um estudo progressivo sobre criação de diretórios, caminhos, Bash, permissões, automação e funcionamento interno do Linux.

Projeto Aprenda Linux BR

Material técnico oficial da playlist dedicada ao comando mkdir

Ambiente principal: Ubuntu Linux • Bash • GNU Coreutils

Como usar este e-book

Este livro foi organizado para acompanhar uma aprendizagem progressiva. Ele começa com o ato mais simples — criar um diretório — e avança somente quando o conceito anterior já fornece base para o próximo. O objetivo não é decorar opções do comando, mas compreender a cadeia de responsabilidades que existe entre o que você digita, o que o Bash interpreta, o programa que é executado e o que o sistema operacional finalmente precisa fazer.

Regra editorial central: Na playlist do mkdir, o mkdir sempre é o protagonista. Outros comandos, operadores e recursos do shell entram para verificar, transformar, navegar, tratar erros ou automatizar uma ação iniciada pelo mkdir.

Ao longo do texto, uma distinção será repetida de propósito: algumas coisas pertencem ao programa mkdir; outras pertencem ao Bash; outras são programas independentes; e outras são mecanismos internos do Linux. Essa separação evita um erro comum de aprendizagem: atribuir ao mkdir comportamentos que, na realidade, foram produzidos pelo shell antes mesmo de o programa começar a executar.

Os exemplos usam comandos em letras minúsculas e, salvo indicação em contrário, assumem uma sessão Bash em um Ubuntu com GNU Coreutils. Quando um comportamento é POSIX, GNU, específico do Bash, dependente do Linux, do filesystem, de ACLs ou de SELinux, isso é indicado explicitamente.

Mapa de aprendizagem

Bloco	Faixa	Foco
Parte I — Fundamentos e composição de comandos	Capítulos 1–8	Criar diretórios, múltiplos operandos, operadores <code>;</code> , <code>&&</code> , <code> </code> , status de saída, <code>cd</code> , <code>pwd</code> , <code>mv</code> e <code>ls</code> .
Parte II — Caminhos e árvores de diretórios	Capítulos 9–17	Caminhos relativos e absolutos, <code>.</code> , <code>..</code> , <code>~</code> , <code>\$HOME</code> , pais inexistentes, <code>-p</code> , árvores e idempotência.
Parte III — Nomes, quoting e expansões do Bash	Capítulos 18–30	<code>-v</code> , espaços, caracteres especiais, ocultos, nomes iniciados por <code>-</code> , regras de nomes, brace expansion, globbing e pipeline de expansões.
Parte IV — Permissões e metadados	Capítulos 31–38	<code>umask</code> , <code>-m</code> , modos numéricos e simbólicos, <code>r/w/x</code> em diretórios, <code>setgid</code> , sticky bit, propriedade e ACL padrão.
Parte V — Falhas e diagnóstico	Capítulos 39–43	<code>File exists</code> , limites do <code>-p</code> , <code>ENOENT</code> , <code>ENOTDIR</code> , <code>EACCES</code> , <code>ENOSPC</code> , <code>EROFS</code> e sucesso parcial.
Parte VI — Variáveis, scripts e automação	Capítulos 44–51	variáveis, quoting, parameter expansion, command substitution, loops, <code>if</code> , parâmetros posicionais e projeto prático.
Parte VII — Portabilidade e funcionamento interno	Capítulos 52–55	POSIX x GNU x Bash, SELinux, caminho do terminal ao kernel, VFS, dentries, inodes e filesystem concreto.

PARTE I — FUNDAMENTOS E COMPOSIÇÃO DE COMANDOS

CAPÍTULO 01 — Comando `mkdir`: criando diretórios no Linux

Objetivo: Entender o que é um diretório, o que o programa `mkdir` faz e como ler sua sintaxe básica.

Fundamento

Um diretório é um objeto do filesystem usado para associar nomes a outros objetos. Na prática, ele organiza a hierarquia em que arquivos e outros diretórios são encontrados. O comando `mkdir` existe para solicitar a criação de um ou mais diretórios a partir de pathnames fornecidos como operandos.

Como funciona

No ambiente GNU, a forma geral pode ser lida como `mkdir [opção]... nome...`. O trecho `nome...` indica que um ou mais nomes podem ser fornecidos. Nesta primeira etapa, porém, interessa apenas o caso mínimo: um único operando e nenhuma opção. O shell lê a linha, localiza o executável `mkdir` e entrega a ele o argumento `projeto`.

Por que isso importa

O programa não “entra” no diretório, não lista seu conteúdo e não muda a posição do usuário no filesystem. Sua responsabilidade aqui é criar o diretório solicitado. Se a operação for bem-sucedida, o GNU `mkdir` normalmente não imprime nada; silêncio, nesse caso, não significa ausência de ação.

Exemplo prático

```
mkdir projeto
```

Resultado esperado

Um novo diretório chamado `projeto` passa a existir no diretório de trabalho atual. Se o nome já existir, o comando comum falha e informa o erro.

Mapa de responsabilidade: `mkdir`: criação do diretório. Bash: interpretação da linha, procura do programa e passagem do operando.

Erro ou cuidado comum: Não confunda “pasta” da interface gráfica com um conceito separado. Em Linux, a GUI normalmente representa um diretório do filesystem.

Base técnica: GNU Coreutils: *mkdir invocation* • POSIX: *mkdir utility*

Conexão com o próximo capítulo

No próximo capítulo, a mesma sintaxe será usada para mostrar que um único processo de `mkdir` pode receber vários operandos.

CAPÍTULO 02 — `mkdir` criando vários diretórios de uma vez

Objetivo: Perceber que múltiplos diretórios podem ser solicitados na mesma invocação, sem loops e sem brace expansion.

Fundamento

Como a sintaxe aceita ``name...``, o programa pode receber vários operandos. Cada palavra entregue pelo shell como argumento separado é tratada como um nome de diretório a ser criado. Isso é uma capacidade do próprio utilitário ``mkdir``, não do brace expansion.

Como funciona

A distinção é importante porque ``mkdir src docs tests`` já resolve uma necessidade comum com portabilidade muito maior do que uma forma específica de Bash. O programa processa os nomes na ordem em que eles aparecem. Se todos forem válidos e puderem ser criados, o status final é de sucesso.

Por que isso importa

Esse comportamento também prepara uma noção que voltará no capítulo sobre sucesso parcial: a execução não é uma transação indivisível. Um diretório anterior pode ter sido criado antes de uma falha ocorrer em um operando posterior.

Exemplo prático

```
mkdir src docs tests
```

Resultado esperado

São criados três diretórios irmãos: ``src``, ``docs`` e ``tests``.

Mapa de responsabilidade: `mkdir`: aceita e processa múltiplos operandos. Bash: separa as palavras e monta a lista de argumentos.

Erro ou cuidado comum: Não ensine ``mkdir {src,docs,tests}`` como se as chaves fossem sintaxe do `mkdir`. Essa forma será estudada depois como brace expansion do Bash.

Base técnica: GNU Coreutils: *mkdir invocation* • POSIX: *mkdir utility*

Conexão com o próximo capítulo

Agora que a criação está clara, entraremos na composição de comandos: primeiro com o operador sequencial ``;`

CAPÍTULO 03 — `mkdir` + `ls` com ``;;``: execução sequencial

Objetivo: Usar ``;` para executar uma verificação depois do `mkdir`, independentemente do sucesso da criação.

Fundamento

O ponto e vírgula é um operador de controle do shell. Ele separa comandos de uma lista e instrui o Bash a executar um, esperar sua conclusão e então seguir para o próximo. Portanto, ``mkdir projeto ; ls`` não é uma forma especial do `mkdir`: são dois comandos independentes organizados pelo shell.

Como funciona

Essa construção é didaticamente útil porque mantém o `mkdir` como ação principal e usa ``ls`` apenas para observar o estado do diretório de trabalho em seguida. Mesmo se ``mkdir projeto``

falhar, o Bash ainda executará `ls`, pois a semântica do `;` não depende do status de sucesso do primeiro comando.

Por que isso importa

O status final de uma lista separada por ponto e vírgula é, em geral, o status do último comando executado. Isso significa que olhar apenas para `$?` depois da linha pode esconder o fato de que o `mkdir` falhou se o `ls` posterior tiver sido bem-sucedido.

Exemplo prático

```
mkdir projeto ; ls
```

Resultado esperado

O shell tenta criar `projeto` e depois executa `ls` em qualquer caso.

Mapa de responsabilidade: `mkdir`: criação. `;`: operador do Bash. `ls`: outro programa, usado apenas para verificar.

Erro ou cuidado comum: Não chame `;` de parâmetro do `mkdir`. Ele é reconhecido pelo parser do shell antes que o executável seja iniciado.

Base técnica: *GNU Bash: Lists of Commands*

Conexão com o próximo capítulo

O próximo passo é alterar a condição: com `&&`, o segundo comando só será executado quando o `mkdir` tiver sucesso.

CAPÍTULO 04 — `mkdir` + `ls` com `&&`: execução condicional

Objetivo: Compreender uma AND list e relacionar a execução do segundo comando ao status zero do `mkdir`.

Fundamento

No Bash, `&&` forma uma lista AND. O segundo comando é executado somente se o primeiro terminar com status zero. Isso permite transformar o resultado do `mkdir` em uma decisão imediata do shell.

Como funciona

Em `mkdir projeto && ls`, primeiro o programa `mkdir` tenta criar o diretório. Se a criação for bem-sucedida, ele termina com status zero e o Bash executa `ls`. Se `projeto` já existir ou ocorrer outra falha, o status será diferente de zero e o `ls` daquela lista não será executado.

Por que isso importa

Esse mecanismo é mais do que um atalho visual. Ele é uma forma elementar de controle de fluxo baseada em status de saída. A mesma ideia aparece em scripts, pipelines de automação e tarefas de provisionamento.

Exemplo prático

```
mkdir projeto && ls
```

Resultado esperado

`ls` aparece somente quando `mkdir projeto` termina com sucesso.

Mapa de responsabilidade: `mkdir`: produz o status de saída. `&&`: Bash decide se continua. `ls`: verificação.

Erro ou cuidado comum: Não diga que `&&` “faz o `mkdir` listar”. O `mkdir` termina primeiro; depois o shell decide se executa um programa diferente.

Base técnica: *GNU Bash: Lists of Commands* • *GNU Coreutils: mkdir exit status*

Conexão com o próximo capítulo

No capítulo seguinte veremos a operação complementar: executar algo apenas quando o `mkdir` falhar.

CAPÍTULO 05 — `mkdir + ||`: executando algo quando ocorre erro

Objetivo: Usar uma OR list do Bash para reagir a falhas reais do `mkdir`.

Fundamento

O operador `||` forma uma lista OR. O comando à direita é executado somente se o comando à esquerda retornar status diferente de zero. Em vez de representar “ou” no sentido de escolher arbitrariamente entre comandos, ele implementa uma decisão baseada no resultado anterior.

Como funciona

Um caso simples é tentar criar um diretório que já existe. `mkdir projeto || echo "não foi possível criar"` faz do `mkdir` o teste real: se ele falhar, o Bash executa o `echo`. O texto não vem do `mkdir`; é uma ação separada de tratamento.

Por que isso importa

Essa estrutura pode ser encadeada com cuidado, mas antes de criar cadeias complexas é melhor dominar a leitura do status. O importante é perceber que o shell não “sabe” por que o `mkdir` falhou; ele recebe apenas o status numérico e, a partir dele, decide se executa o ramo à direita.

Exemplo prático

```
mkdir projeto || echo "não foi possível criar o diretório"
```

Resultado esperado

Se o `mkdir` falhar, a mensagem adicional é impressa pelo builtin `echo` do Bash.

Mapa de responsabilidade: `mkdir`: tenta criar e retorna status. `||`: controle do Bash. `echo`: builtin independente.

Erro ou cuidado comum: `||` não identifica a causa específica do erro. Para diagnosticar a causa, observe a mensagem do `mkdir` e, em níveis mais avançados, as condições/errores subjacentes.

Base técnica: *GNU Bash: Lists of Commands* • *GNU Bash: Exit Status*

Conexão com o próximo capítulo

Agora veremos explicitamente os status zero, não zero e o parâmetro especial `$?` .

CAPÍTULO 06 — `mkdir` e status de saída

Objetivo: Entender como o sucesso e a falha do `mkdir` são convertidos em um valor que o shell pode usar.

Fundamento

Programas de linha de comando comunicam ao processo chamador um status de término. Para o Bash, status zero representa sucesso; um valor diferente de zero representa falha. O GNU `mkdir` segue essa convenção: zero quando a invocação termina com sucesso e não zero quando há falha.

Como funciona

O Bash guarda o status do comando mais recentemente executado no parâmetro especial `$?` . Por isso, após `mkdir projeto`, executar `echo $?` permite observar a decisão de sucesso ou falha. É importante fazer a consulta imediatamente: qualquer comando executado depois substitui o valor anterior.

Por que isso importa

O número não deve ser confundido com `errno`. Um utilitário pode encontrar diferentes erros do sistema e ainda mapear sua execução para um status de saída próprio. O `errno` pertence à interface de chamadas de sistema/biblioteca; o status observado pelo shell é a convenção do processo que terminou.

Exemplo prático

```
mkdir projeto
echo $?
```

Resultado esperado

Em uma criação bem-sucedida, o segundo comando normalmente imprime `0`. Em caso de falha, imprime um valor não zero.

Mapa de responsabilidade: `mkdir`: determina seu status de término. Bash: expõe o último status em `$?` . `echo`: mostra o valor.

Erro ou cuidado comum: Consultar `$?` depois de `ls`, `echo` ou qualquer outro comando mostra o status desse comando mais recente, não o do `mkdir` anterior.

Base técnica: GNU Bash: *Exit Status* • GNU Coreutils: *mkdir exit status*

Conexão com o próximo capítulo

Com status de saída entendido, podemos usar o resultado da criação para navegar no diretório e confirmar a localização.

CAPÍTULO 07 — mkdir + cd + pwd

Objetivo: Criar um diretório, entrar nele e identificar o diretório de trabalho atual, distinguindo programas externos de builtins.

Fundamento

O `cd` é um builtin do shell porque precisa alterar o diretório de trabalho do próprio processo Bash. Um programa externo filho não conseguiria mudar permanentemente o diretório de trabalho do shell pai. `pwd` também existe como builtin no Bash, embora possa existir como utilitário externo em sistemas GNU.

Como funciona

Em uma sequência como `mkdir projeto && cd projeto && pwd`, o `mkdir` cria; o primeiro `&&` exige sucesso para que o `cd` seja tentado; o segundo exige que a mudança de diretório tenha funcionado antes de mostrar a localização. O diretório de trabalho influencia como caminhos relativos serão resolvidos nos próximos capítulos.

Por que isso importa

Essa é uma boa demonstração de responsabilidades: o `mkdir` altera o filesystem, o `cd` altera o estado interno do shell e o `pwd` informa o caminho corrente.

Exemplo prático

```
mkdir projeto && cd projeto && pwd
```

Resultado esperado

Se tudo funcionar, a saída do `pwd` termina em `/projeto` e o shell permanece dentro desse diretório após a linha.

Mapa de responsabilidade: `mkdir`: programa externo de criação. `cd` e `pwd`: builtins do Bash neste contexto. `&&`: operadores do shell.

Erro ou cuidado comum: Evite ensinar `cd` como “continuação” do `mkdir`. O `cd` atua no processo do shell e tem função completamente diferente.

Base técnica: *GNU Bash: Bourne Shell Builtins* • *GNU Bash: Command Search and Execution*

Conexão com o próximo capítulo

No próximo capítulo, veremos outra operação posterior à criação: renomear com `mv` e verificar com `ls`.

CAPÍTULO 08 — mkdir + mv + ls

Objetivo: Criar um diretório, renomeá-lo e verificar o resultado sem perder a separação entre os programas.

Fundamento

O `mv` é outro utilitário, usado para mover ou renomear objetos. Quando o destino está no mesmo filesystem e a operação é um simples rename, o comportamento interno pode ser eficiente, mas isso não muda o ponto didático: `mv` não é uma opção do `mkdir`.

Como funciona

Na linha `mkdir projeto && mv projeto site && ls`, a primeira ação cria `projeto`. Só depois, se houver sucesso, `mv` tenta renomeá-lo para `site`; então `ls` verifica o estado final. Essa ordem deixa visível a construção por etapas.

Por que isso importa

Comandos compostos assim devem ser usados para demonstrar causalidade, não para esconder conceitos. O aluno precisa conseguir apontar qual componente fez cada alteração.

Exemplo prático

```
mkdir projeto && mv projeto site && ls
```

Resultado esperado

Ao final, o nome `site` deve aparecer; `projeto` deixa de existir com esse nome.

Mapa de responsabilidade: `mkdir`: cria. `mv`: move/renomeia. `ls`: lista. `&&`: Bash controla a sequência condicional.

Erro ou cuidado comum: Se já existir um destino chamado `site`, o comportamento de `mv` depende do tipo do destino e de opções; isso pertence ao estudo de `mv`, não ao `mkdir`.

Base técnica: *GNU Bash: Lists of Commands* • *GNU Coreutils: mv*

Conexão com o próximo capítulo

A partir daqui, o foco muda para pathnames: o mesmo `mkdir` produzirá resultados diferentes conforme o caminho entregue a ele.

PARTE II — CAMINHOS E ÁRVORES DE DIRETÓRIOS

CAPÍTULO 09 — `mkdir` com caminho relativo

Objetivo: Entender que um pathname relativo é resolvido a partir do diretório de trabalho atual.

Fundamento

Um pathname que não começa com ``/`` é relativo. No Linux, a resolução de um caminho relativo começa no diretório de trabalho atual do processo. Quando você executa ``mkdir documentos/relatorios``, o `mkdir` entrega ao sistema um caminho cujo ponto inicial depende de onde o processo está.

Como funciona

Isso explica por que o mesmo comando textual pode criar objetos em lugares diferentes em sessões distintas. Se o shell estiver em ``/home/aluno``, ``documentos/relatorios`` parte dali; se estiver em ``/tmp``, parte de ``/tmp``. O texto do operando não contém a localização completa.

Por que isso importa

Antes de ``-p``, todos os componentes intermediários precisam existir e ser diretórios pesquisáveis. A resolução percorre cada componente até chegar ao nome final que será criado.

Exemplo prático

```
mkdir documentos
```

Resultado esperado

Se o diretório de trabalho for ``/home/aluno``, o novo objeto será ``/home/aluno/documentos``.

Mapa de responsabilidade: `mkdir`: recebe o pathname relativo. Kernel/VFS: resolve o pathname a partir do cwd do processo.

Erro ou cuidado comum: Não descreva caminho relativo como “caminho curto”. O critério técnico é começar ou não com ``/``.

Base técnica: *Linux man-pages: path_resolution(7)* • *Linux man-pages: chdir(2)*

Conexão com o próximo capítulo

O próximo capítulo introduz ``.`` como referência explícita ao diretório atual.

CAPÍTULO 10 — `mkdir` usando ``.`

Objetivo: Compreender ``.`` como referência ao diretório atual e usar ``./`` para tornar explícito o ponto de partida.

Fundamento

Por convenção, ``.`` representa o próprio diretório atual. Em ``mkdir ./projeto``, o componente ``.`` não cria nada novo; ele diz que a resolução deve continuar a partir do diretório atual e então criar o componente final ``projeto``.

Como funciona

Em muitos casos, ``mkdir projeto`` e ``mkdir ./projeto`` produzem o mesmo resultado. A segunda forma, porém, pode ser útil para tornar explícito que um nome deve ser tratado como pathname relativo. Ela também ajuda a lidar com nomes iniciados por hífen, como veremos mais adiante.

Por que isso importa

Na resolução de pathnames, o significado de ``.`` é convencional. O kernel trata esse componente como o diretório em que a busca já se encontra.

Exemplo prático

```
mkdir ./projeto
```

Resultado esperado

Cria ``projeto`` no diretório atual.

Mapa de responsabilidade: `mkdir`: recebe ``.`./projeto``. Resolução de pathname: interpreta ``.`` como diretório atual.

Erro ou cuidado comum: ``.`/`` não é uma opção do `mkdir` e não significa “executar o `mkdir` daqui”. Ele faz parte do pathname operando.

Base técnica: *Linux man-pages: path_resolution(7)*

Conexão com o próximo capítulo

Depois do diretório atual, estudaremos ``.`..``, que referencia o diretório pai.

CAPÍTULO 11 — `mkdir` usando ``.`..``

Objetivo: Usar ``.`..`` em caminhos relativos e compreender a referência ao diretório pai.

Fundamento

O componente ``.`..`` representa o diretório pai no processo de resolução de pathname. Se você estiver em ``/home/aluno/projeto``, o operando ``.`../backup`` normalmente aponta para ``/home/aluno/backup``.

Como funciona

Essa resolução é feita no contexto da hierarquia e pode interagir com links simbólicos e com a forma lógica/física usada pelo shell em operações como ``cd``. Para o `mkdir`, o que importa é o pathname efetivamente entregue e resolvido pelo sistema.

Por que isso importa

Na raiz, não é possível subir acima da própria raiz: ``.`..`` é tratado como ``.`/``. Essa regra ajuda a manter a resolução confinada à raiz do processo.

Exemplo prático

```
mkdir ../backup
```

Resultado esperado

Cria ``backup`` no diretório pai do diretório de trabalho atual, desde que haja permissão e o pai exista.

Mapa de responsabilidade: `mkdir`: solicita a criação. ``.``: componente do pathname interpretado durante a resolução.

Erro ou cuidado comum: Evite ensinar ``.`` apenas como “voltar uma pasta”. Em pathname, ele é um componente que referencia o pai durante a resolução.

Base técnica: *Linux man-pages: path_resolution(7)*

Conexão com o próximo capítulo

Agora compararemos essa resolução dependente do `cwd` com um caminho absoluto, que começa na raiz.

CAPÍTULO 12 — `mkdir` com caminho absoluto

Objetivo: Entender que pathnames iniciados por ``/`` são resolvidos a partir da raiz do processo.

Fundamento

Um pathname absoluto começa com ``/``. Nesse caso, o ponto inicial da resolução deixa de ser o diretório de trabalho atual e passa a ser a raiz vista pelo processo. Assim, ``/tmp/laboratorio`` aponta para o mesmo local independentemente de você estar em ``/home``, ``/var`` ou outro diretório.

Como funciona

Isso não significa que qualquer caminho absoluto seja gravável. A criação ainda depende da existência dos componentes intermediários, permissões de busca nos diretórios do caminho e permissão de escrita no diretório pai final.

Por que isso importa

Em ambientes com namespaces, chroot ou containers, a “raiz do processo” pode não ser a mesma visão global que outro processo possui. Para o aprendizado inicial, ``/`` é a raiz da hierarquia visível naquela sessão/processo.

Exemplo prático

```
mkdir /tmp/laboratorio
```

Resultado esperado

Cria ``/tmp/laboratorio`` se ``/tmp`` existir e a política de permissões permitir.

Mapa de responsabilidade: `mkdir`: recebe o pathname. Kernel/VFS: inicia a busca na raiz do processo porque o caminho começa com ``/``.

Erro ou cuidado comum: Caminho absoluto não significa “caminho permitido” nem “caminho real universal”. Ele apenas define o ponto inicial da resolução.

Base técnica: *Linux man-pages: path_resolution(7)*

Conexão com o próximo capítulo

O próximo capítulo contrasta duas formas do Bash de chegar ao diretório pessoal: `~` e `$HOME`.

CAPÍTULO 13 — `mkdir` com `~` e `$HOME`

Objetivo: Diferenciar tilde expansion de parameter expansion e observar que ambas acontecem no shell.

Fundamento

Quando você digita `mkdir ~/projeto`, o caractere `~` não é interpretado pelo `mkdir`. Em Bash, um tilde não citado no início de uma palavra pode sofrer tilde expansion e ser substituído pelo diretório pessoal. Já em `mkdir "$HOME/projeto"`, `$HOME` é parameter expansion: o shell substitui o valor da variável `HOME`.

Como funciona

As duas formas frequentemente apontam para o mesmo local, mas pertencem a mecanismos distintos. A expansão de tilde tem regras próprias; a expansão de parâmetro é o mecanismo geral usado por variáveis e parâmetros do shell.

Por que isso importa

Esse capítulo é um ponto de virada: o texto digitado pelo usuário pode ser transformado pelo Bash antes que o executável `mkdir` receba seus argumentos. O programa pode nunca enxergar o caractere `~` nem a sequência literal `$HOME`.

Exemplo prático

```
mkdir ~/projeto
mkdir "$HOME/documentos"
```

Resultado esperado

O Bash expande cada forma e passa pathnames resultantes ao `mkdir`.

Mapa de responsabilidade: Bash: tilde expansion e parameter expansion. `mkdir`: apenas recebe os argumentos já expandidos.

Erro ou cuidado comum: Colocar `~` entre aspas, como em `mkdir "~/projeto"`, impede a tilde expansion e cria um pathname literal relativo contendo `~` como primeiro componente, se tal componente existir/for criável.

Base técnica: *GNU Bash: Tilde Expansion* • *GNU Bash: Shell Parameter Expansion*

Conexão com o próximo capítulo

Antes de explorar mais expansões, precisamos entender uma falha estrutural comum: diretório pai inexistente.

CAPÍTULO 14 — erro de diretório pai inexistente

Objetivo: Entender por que o `mkdir` comum não cria automaticamente componentes intermediários ausentes.

Fundamento

Suponha que ``projeto`` ainda não exista e você execute ``mkdir projeto/src``. Para criar ``src``, o sistema precisa primeiro resolver ``projeto`` como um diretório existente. Como esse componente intermediário não é encontrado, a resolução falha antes de chegar ao nome final.

Como funciona

No nível de sistema, essa condição está relacionada a ``ENOENT``: um componente do caminho não existe. O programa GNU `mkdir` transforma a falha em uma mensagem diagnóstica para o usuário e termina com status não zero.

Por que isso importa

Esse comportamento é deliberado. Sem uma opção específica, o `mkdir` não presume que você deseja criar toda a cadeia. Ele tenta criar o diretório nomeado pelo operando, e o caminho até o pai precisa ser resolvível.

Exemplo prático

```
mkdir projeto/src
```

Resultado esperado

Se ``projeto`` não existir, ocorre erro e ``src`` não é criado.

Mapa de responsabilidade: `mkdir`: solicita a criação do pathname final. Kernel/VFS: falha ao resolver um componente pai ausente.

Erro ou cuidado comum: A mensagem textual pode variar com locale e implementação. O conceito importante é a condição: componente intermediário inexistente.

Base técnica: *Linux man-pages: mkdir(2)* • *Linux man-pages: path_resolution(7)*

Conexão com o próximo capítulo

A opção ``-p`` existe justamente para lidar com componentes pais ausentes.

CAPÍTULO 15 — `mkdir -p`

Objetivo: Usar ``--parents`` para criar componentes intermediários ausentes e tolerar diretórios já existentes.

Fundamento

Com GNU/POSIX ``mkdir -p``, o utilitário passa a criar componentes intermediários ausentes do pathname. Em ``mkdir -p projeto/src``, se ``projeto`` não existir, ele é criado antes de ``src``. Se um componente já existir como diretório, ele pode ser reutilizado sem erro.

Como funciona

A opção também muda o tratamento do operando final quando ele já existe como diretório: esse caso é ignorado sem erro. Isso torna ``-p`` apropriado para preparar estruturas que podem já estar parcialmente presentes.

Por que isso importa

``p`` não significa “forçar qualquer caminho”. A resolução ainda falha diante de permissões insuficientes, filesystem somente leitura, falta de espaço ou quando um componente que deveria ser diretório é um arquivo regular.

Exemplo prático

```
mkdir -p projeto/src
```

Resultado esperado

Cria ``projeto`` se necessário e, em seguida, cria ``src`` dentro dele.

Mapa de responsabilidade: `mkdir -p`: lógica do utilitário para pais ausentes/existentes. Sistema: continua responsável pelas operações de criação e resolução.

Erro ou cuidado comum: Não apresente ``p`` como solução universal. Ele resolve a ausência de diretórios intermediários, não conflitos de tipo nem restrições do sistema.

Base técnica: GNU Coreutils: *mkdir invocation* • POSIX: *mkdir utility*

Conexão com o próximo capítulo

No próximo capítulo, aplicaremos ``p`` a árvores maiores.

CAPÍTULO 16 — `mkdir -p` criando árvores

Objetivo: Construir hierarquias de vários níveis em uma única invocação.

Fundamento

Uma árvore é apenas uma sequência de componentes relacionados por ``/``. Com ``p``, o `mkdir` pode garantir a existência de todos os diretórios necessários até os operandos finais.

Como funciona

Como o programa também aceita múltiplos operandos, é possível criar ramos diferentes na mesma invocação. A forma mais portátil é escrever cada pathname explicitamente. Mais adiante, brace expansion poderá reduzir a repetição no Bash, mas é importante perceber que a capacidade de processar múltiplos operandos já pertence ao `mkdir`.

Por que isso importa

Criar árvores com ``p`` é comum em scripts de instalação, inicialização de aplicações e preparação de ambientes. A confiabilidade vem de declarar o estado estrutural desejado, sem exigir que cada nível seja criado manualmente em comandos separados.

Exemplo prático

```
mkdir -p projeto/src projeto/docs projeto/tests
```

Resultado esperado

Cria ``projeto`` e os três subdiretórios, reutilizando componentes já existentes que sejam diretórios.

Mapa de responsabilidade: `mkdir: -p` e múltiplos operandos. Bash: apenas entrega os argumentos como escritos.

Erro ou cuidado comum: Se um dos operandos falhar, os anteriores podem já ter sido criados. Isso será detalhado no capítulo 43.

Base técnica: *GNU Coreutils: mkdir invocation* • *POSIX: mkdir utility*

Conexão com o próximo capítulo

A possibilidade de executar a mesma preparação mais de uma vez leva ao conceito de idempotência.

CAPÍTULO 17 — execução repetida de `mkdir -p`

Objetivo: Discutir idempotência com precisão e reconhecer seus limites.

Fundamento

Uma operação idempotente é aquela cuja repetição produz o mesmo estado relevante após a primeira aplicação bem-sucedida. `mkdir -p projeto/src` se aproxima desse comportamento quando o objetivo é apenas garantir que esses diretórios existam: a primeira execução pode criá-los; execuções posteriores, se eles já existirem como diretórios adequados, terminam sem erro por esse motivo.

Como funciona

Isso não torna qualquer uso de `mkdir -p` universalmente idempotente em todos os aspectos. Permissões podem não ser reajustadas em diretórios já existentes; ACLs, ownership e labels de segurança também não são “reaplicados” automaticamente pelo simples fato de usar `-p`. O estado desejado precisa ser definido com precisão.

Por que isso importa

Em automação, a pergunta correta é: “qual propriedade estou tentando garantir?” Para existência da estrutura, `-p` é excelente. Para convergir metadados completos, outras operações podem ser necessárias.

Exemplo prático

```
mkdir -p projeto/src
mkdir -p projeto/src
```

Resultado esperado

A segunda execução não falha apenas porque `projeto/src` já existe como diretório.

Mapa de responsabilidade: `mkdir: -p` tolera diretórios existentes. Conceito de idempotência: propriedade da operação em relação ao estado que está sendo garantido.

Erro ou cuidado comum: Não diga que `-p` “não faz nada” na segunda execução em todos os cenários; ele ainda precisa processar argumentos, verificar componentes e pode falhar por outras causas.

Base técnica: *GNU Coreutils: mkdir invocation*

Conexão com o próximo capítulo

Depois da estrutura, veremos uma opção GNU útil para observar quais diretórios foram efetivamente criados: `-v`.

PARTE III — NOMES, QUOTING E EXPANSÕES DO BASH

CAPÍTULO 18 — `mkdir -v`

Objetivo: Usar o modo verbose do GNU `mkdir` e identificar sua falta de obrigatoriedade no POSIX.

Fundamento

A opção `-v` (`--verbose`) pede ao GNU `mkdir` que imprima uma mensagem para cada diretório criado. Ela é especialmente útil com `-p`, porque permite observar quais componentes estavam ausentes e precisaram ser criados.

Como funciona

Esse recurso é uma extensão GNU. A especificação POSIX de `mkdir` exige `-p` e `-m`, mas não exige `-v`. Portanto, scripts que dependem de `-v` não devem ser tratados como portáteis para qualquer implementação POSIX.

Por que isso importa

Verbose é observabilidade, não uma mudança fundamental na criação. O programa continua criando os diretórios; apenas adiciona saída informativa.

Exemplo prático

```
mkdir -pv projeto/src/modulos
```

Resultado esperado

O GNU `mkdir` imprime mensagens para os diretórios que precisou criar. Componentes já existentes não geram a mesma mensagem de criação.

Mapa de responsabilidade: `mkdir` GNU: `-v` e `-p`. Bash: apenas passa as opções.

Erro ou cuidado comum: Não use `-v` como mecanismo de teste lógico. Para automação, status de saída é mais confiável do que interpretar texto de diagnóstico.

Base técnica: GNU Coreutils: *mkdir invocation* • POSIX: *mkdir utility*

Conexão com o próximo capítulo

Agora entraremos em um conjunto de problemas de nomes: espaços, caracteres especiais e quoting.

CAPÍTULO 19 — nomes com espaços

Objetivo: Entender como o shell transforma espaços em separadores de palavras e como quoting preserva um único argumento.

Fundamento

Em `mkdir meu projeto`, o Bash normalmente reconhece duas palavras de argumento: `meu` e `projeto`. Como o `mkdir` aceita múltiplos operandos, ele tentará criar dois diretórios. O espaço não “entra” em nenhum nome.

Como funciona

Para criar um único nome contendo espaço, é preciso impedir que o shell use esse espaço como separador sintático. Uma forma é usar aspas: `mkdir "meu projeto"`. O quoting pertence ao shell; quando o `mkdir` recebe o argumento, as aspas já foram removidas e o conteúdo do argumento contém o caractere de espaço.

Por que isso importa

Essa distinção explica por que “o comando aceita espaço” é uma formulação incompleta. O filesystem pode aceitar o caractere no nome, mas a linguagem do shell exige que o usuário represente corretamente esse argumento.

Exemplo prático

```
mkdir meu projeto
mkdir "meu projeto"
```

Resultado esperado

A primeira linha tenta criar dois diretórios; a segunda tenta criar um diretório cujo nome contém espaço.

Mapa de responsabilidade: Bash: tokenização/quoting. `mkdir`: processa a quantidade de operandos resultante.

Erro ou cuidado comum: Evite nomes com espaços em exemplos de scripts sem quoting; isso cria hábitos frágeis que reaparecem em variáveis e command substitution.

Base técnica: *GNU Bash: Quoting* • *GNU Coreutils: mkdir invocation*

Conexão com o próximo capítulo

No próximo capítulo, ampliaremos a ideia para metacaracteres e caracteres especiais do shell.

CAPÍTULO 20 — caracteres especiais do shell

Objetivo: Reconhecer quando quoting ou escaping são necessários para impedir interpretações do Bash.

Fundamento

O shell atribui significado especial a caracteres como ```, `?`, `$`, `;`, `&`, `|`, parênteses, redirecionadores e espaços, dependendo do contexto. Se esses caracteres fizerem parte literal do nome desejado, o Bash precisa ser instruído a tratá-los como dados.

Como funciona

Aspas simples preservam praticamente todo o conteúdo literalmente; aspas duplas preservam espaços e muitos caracteres, mas ainda permitem certas expansões, como `$variavel` e `$(...)`. A barra invertida pode proteger um caractere individual. O objetivo não é decorar todos os casos de uma vez, e sim entender a fronteira: quoting é processamento do shell antes do `mkdir`.

Por que isso importa

Exemplos com nomes extravagantes servem apenas para revelar a regra. Em ambientes reais, nomes simples reduzem ambiguidades, mas scripts robustos não devem presumir que entradas externas nunca conterão espaços ou metacaracteres.

Exemplo prático

```
mkdir 'relatorio;final'
mkdir "preco\$2026"
mkdir arquivo\?
```

Resultado esperado

São solicitados nomes literais contendo `;`, `\$` ou `?`, em vez de acionar os significados especiais desses caracteres no shell.

Mapa de responsabilidade: Bash: quoting/escaping. `mkdir`: recebe os caracteres literais no argumento.

Erro ou cuidado comum: Quoting incorreto pode transformar parte do nome em operador, expansão ou múltiplos argumentos antes de o `mkdir` começar.

Base técnica: *GNU Bash: Quoting*

Conexão com o próximo capítulo

Um caso visualmente simples, mas conceitualmente importante, são os nomes iniciados por ponto.

CAPÍTULO 21 — diretórios ocultos

Objetivo: Entender por que nomes iniciados por ponto são tratados como ocultos por convenções de ferramentas, não por um atributo especial de filesystem.

Fundamento

Criar `.configuracao` com `mkdir` não aciona uma “opção oculto” do filesystem. O ponto é simplesmente o primeiro caractere do nome. Ferramentas como `ls` omitem, por padrão, entradas cujo nome começa com ponto, seguindo uma convenção histórica de Unix.

Como funciona

Isso explica por que o diretório existe normalmente, participa de permissões e resolução de pathname e pode ser acessado como qualquer outro nome válido. Para visualizá-lo com GNU `ls`, costuma-se usar uma opção como `-a`, mas essa opção pertence ao `ls`, não ao `mkdir`.

Por que isso importa

A convenção de ocultação é um bom exemplo de como comportamento de interface pode ser confundido com metadado do filesystem. Aqui não há um “bit hidden” semelhante ao modelo de alguns outros sistemas.

Exemplo prático

```
mkdir .configuracao && ls -a
```

Resultado esperado

O `mkdir` cria `.configuracao`; `ls -a` mostra a entrada que um `ls` comum poderia omitir.

Mapa de responsabilidade: `mkdir`: cria o nome literal. `ls`: decide o que exibir por padrão/opção.

Erro ou cuidado comum: Não descreva o ponto como “atributo oculto” armazenado separadamente do nome.

Base técnica: *GNU Coreutils: ls • Linux pathname conventions*

Conexão com o próximo capítulo

Outro problema de nomes ocorre quando o primeiro caractere é ```, pois programas de linha de comando podem interpretar o operando como opção.

CAPÍTULO 22 — nomes iniciados por -

Objetivo: Distinguir opções de operandos e usar `--`` ou um pathname iniciado por ``./`` para desambiguar.

Fundamento

Na linha de comando, argumentos iniciados por hífen costumam ser interpretados como opções. Portanto, ``mkdir -projeto`` pode ser analisado como uma combinação de opções inválidas, não como um nome.

Como funciona

GNU Coreutils oferece a convenção `--`` para encerrar a lista de opções: ``mkdir -- -projeto``. Tudo depois desse marcador é tratado como operando. Outra solução é tornar o nome um pathname que não começa por hífen, como ``mkdir ./-projeto``.

Por que isso importa

As duas formas mostram novamente que representação na linha de comando e nome no filesystem são coisas diferentes. O diretório criado continua se chamando ``-projeto``; ``./`` serve apenas para formar o pathname usado para alcançá-lo.

Exemplo prático

```
mkdir -- -projeto
mkdir ./-outro
```

Resultado esperado

Cria diretórios chamados ``-projeto`` e ``-outro`` no diretório atual.

Mapa de responsabilidade: GNU common option parsing: `--``. Resolução de pathname: ``./``. `mkdir`: cria o operando final.

Erro ou cuidado comum: Não use `--`` como se fosse uma opção funcional de criação; ele é um delimitador de opções comum a muitos utilitários GNU.

Base técnica: *GNU Coreutils: Common options • Linux man-pages: pathname(7)*

Conexão com o próximo capítulo

No próximo capítulo, consolidaremos o que realmente pode compor um nome e por que limites variam por filesystem.

CAPÍTULO 23 — nomes de diretórios no Linux

Objetivo: Compreender componentes de pathname, separador ``/``, NUL, limites e dependência do filesystem.

Fundamento

Um pathname é composto por componentes separados por ``/``. No Linux, a interface de pathname usa strings terminadas por byte NUL; por isso o byte NUL não pode aparecer dentro de um nome passado por essas interfaces. A barra ``/`` é o separador de componentes e também não pode fazer parte de um único componente de nome.

Como funciona

Fora essas regras fundamentais da interface, limites e restrições adicionais podem depender do filesystem. É comum encontrar ``NAME_MAX`` de 255 bytes em filesystems populares, mas isso não deve ser transformado em regra universal. O valor aplicável pode ser consultado com interfaces como ``pathconf()`` ou utilitários como ``getconf`` para um local específico.

Por que isso importa

Também há diferença entre tamanho em bytes e quantidade de caracteres visíveis, especialmente com codificações multibyte. E um filesystem pode impor regras adicionais de normalização, codificação ou caracteres. Por isso, “Linux aceita qualquer coisa menos `/` e NUL” é uma aproximação útil apenas se acompanhada da ressalva de dependência do filesystem e da interface.

Exemplo prático

```
getconf NAME_MAX .
getconf PATH_MAX .
mkdir "nome-invalido"
```

Resultado esperado

``getconf`` mostra limites associados ao caminho consultado; o `mkdir` usa o nome solicitado dentro dessas restrições.

Mapa de responsabilidade: Filesystem/kernel: limites e regras de nome. `getconf`: consulta. `mkdir`: tenta criar conforme o pathname recebido.

Erro ou cuidado comum: Não copie o limite do ext4 para todos os filesystems. Nem confunda caractere Unicode com byte.

Base técnica: *Linux man-pages: pathname(7)* • *Linux man-pages: pathconf(3)* • *POSIX: pathname definition*

Conexão com o próximo capítulo

Agora voltaremos ao Bash para estudar uma expansão que gera múltiplos argumentos antes de executar mkdir: brace expansion.

CAPÍTULO 24 — brace expansion

Objetivo: Usar brace expansion como geração textual do Bash e separá-la da capacidade nativa de múltiplos operandos do mkdir.

Fundamento

Em Bash, ``mkdir {src,docs,tests}`` passa primeiro por brace expansion. O shell gera as palavras ``src``, ``docs`` e ``tests`` antes de procurar/executar o programa. O mkdir recebe três operandos comuns; ele não recebe as chaves.

Como funciona

A expansão é estritamente textual e ocorre antes das demais expansões. Os nomes gerados não precisam existir previamente. Essa característica a diferencia do globbing, que procura correspondências em nomes existentes.

Por que isso importa

Didaticamente, vale comparar ``mkdir src docs tests`` com ``mkdir {src,docs,tests}``. O primeiro depende apenas da capacidade do mkdir de aceitar múltiplos operandos; o segundo adiciona uma conveniência específica de shells que implementam brace expansion, como Bash.

Exemplo prático

```
mkdir {src,docs,tests}
```

Resultado esperado

O Bash transforma a expressão em três argumentos e o mkdir tenta criar os três diretórios.

Mapa de responsabilidade: Bash: brace expansion. mkdir: múltiplos operandos resultantes.

Erro ou cuidado comum: Brace expansion não é exigida pelo POSIX shell e não deve ser descrita como opção ou sintaxe do mkdir.

Base técnica: GNU Bash: Brace Expansion

Conexão com o próximo capítulo

No próximo capítulo, veremos como prefixos e sufixos se combinam com cada item das chaves.

CAPÍTULO 25 — prefixos e sufixos com brace expansion

Objetivo: Explorar a concatenação textual que torna brace expansion útil para famílias de nomes.

Fundamento

Brace expansion preserva texto antes e depois das chaves. Em ``mkdir modulo-{api,web,worker}``, o prefixo ``modulo-`` é anexado a cada alternativa, produzindo três palavras. Da mesma forma, ``mkdir {dev,prod}-config`` acrescenta o sufixo ``-config``.

Como funciona

Esse mecanismo reduz repetição na linha digitada, mas não muda a semântica do mkdir. Após a expansão, o programa vê apenas uma lista normal de strings. Se uma delas já existir ou for inválida naquele contexto, a falha pertence à criação desse operando.

Por que isso importa

Expansões podem ser aninhadas, mas linhas excessivamente compactas podem se tornar mais difíceis de revisar do que uma lista explícita. Em ensino e automação, concisão não deve vencer clareza.

Exemplo prático

```
mkdir modulo-{api,web,worker}
mkdir {dev,prod}-config
```

Resultado esperado

São gerados `modulo-api`, `modulo-web`, `modulo-worker`, `dev-config` e `prod-config` nas respectivas invocações.

Mapa de responsabilidade: Bash: gera textos com prefixo/sufixo. mkdir: cria os operandos.

Erro ou cuidado comum: Não assuma que a expansão “verifica” os nomes. Ela gera strings; a validade/possibilidade de criação é avaliada depois.

Base técnica: GNU Bash: Brace Expansion

Conexão com o próximo capítulo

A mesma expansão suporta sequências numéricas e alfabéticas, úteis para séries previsíveis.

CAPÍTULO 26 — sequências com brace expansion

Objetivo: Gerar séries numéricas com preenchimento por zero e entender que a sequência é produzida no Bash.

Fundamento

Uma expressão como `{01..10}` é uma sequência de brace expansion do Bash. Como os limites começam com zero, o Bash preserva a largura e gera `01`, `02` até `10`. Ao ser concatenada com `aula-`, cada termo forma um argumento independente.

Como funciona

Isso é diferente de um loop: o Bash gera a lista de palavras durante a expansão e depois executa uma única invocação do mkdir com vários operandos. Um loop, por outro lado, pode executar o mkdir repetidamente e aplicar lógica por iteração.

Por que isso importa

As sequências podem ter incremento explícito, como `{00..20..5}`. Ainda assim, o mecanismo é textual; não há consulta ao filesystem durante a geração.

Exemplo prático

```
mkdir aula-{01..10}
```

Resultado esperado

O Bash gera dez operandos (``aula-01`` a ``aula-10``) e o `mkdir` os processa em ordem.

Mapa de responsabilidade: Bash: sequência de brace expansion. `mkdir`: uma invocação com múltiplos operandos.

Erro ou cuidado comum: Não confunda zero padding da expansão com um recurso de formatação do `mkdir`.

Base técnica: *GNU Bash: Brace Expansion*

Conexão com o próximo capítulo

Combinaremos agora brace expansion com ``p`` para formar árvores maiores.

CAPÍTULO 27 — `mkdir -p` com brace expansion

Objetivo: Criar ramos estruturados combinando uma opção real do `mkdir` com geração textual do Bash.

Fundamento

``mkdir -p projeto/{src,docs,tests}`` reúne dois mecanismos independentes. Primeiro, o Bash expande as chaves e produz ``projeto/src``, ``projeto/docs`` e ``projeto/tests``. Depois, o `mkdir` recebe ``p`` e esses três operandos.

Como funciona

Com ``p``, o componente ``projeto`` pode ser criado durante o processamento do primeiro operando e reutilizado nos seguintes. A ordem de processamento continua existindo, mas o estado criado por um operando pode facilitar os posteriores.

Por que isso importa

Essa combinação é poderosa para estruturas previsíveis, desde que a equipe saiba que o script depende do Bash (ou de outro shell compatível com brace expansion). Se portabilidade POSIX for requisito, escreva os operandos explicitamente ou use outro mecanismo especificado.

Exemplo prático

```
mkdir -p projeto/{src,docs,tests}
```

Resultado esperado

Cria a raiz ``projeto`` e os três ramos, se ausentes.

Mapa de responsabilidade: Bash: `{src,docs,tests}``. `mkdir`: ``p`` e criação dos pathnames resultantes.

Erro ou cuidado comum: A linha parece uma única “sintaxe `mkdir`”, mas é o resultado da composição entre linguagem do shell e utilitário.

Base técnica: *GNU Bash: Brace Expansion* • *GNU Coreutils: `mkdir` invocation*

Conexão com o próximo capítulo

O próximo capítulo contrasta brace expansion com globbing, duas expansões que frequentemente são confundidas.

CAPÍTULO 28 — brace expansion versus globbing

Objetivo: Diferenciar geração textual independente do filesystem de filename expansion baseada em correspondências existentes.

Fundamento

Brace expansion gera strings por regra textual. `{a,b}` produz `a` e `b` mesmo que nenhum desses nomes exista. Globbing, ou filename expansion, interpreta padrões como `*`, `?` e classes entre colchetes para encontrar pathnames existentes que correspondam ao padrão.

Como funciona

A ordem também importa: brace expansion ocorre antes de filename expansion. Uma expressão pode gerar palavras que posteriormente ainda contenham padrões de globbing. Esse pipeline explica por que o resultado final entregue ao `mkdir` pode ter sido transformado em mais de uma fase.

Por que isso importa

Para criação, brace expansion é naturalmente útil porque gera nomes novos. Globbing é mais comum para selecionar objetos que já existem. Por isso, usar `mkdir projeto-*` como tentativa de “criar vários projetos” é conceitualmente errado: se houver correspondências, o padrão vira nomes existentes; se não houver, o comportamento depende de opções do shell como `nullglob`, e o padrão pode permanecer literal.

Exemplo prático

```
mkdir {alpha,beta}
# contraste conceitual: * procura correspondências existentes
```

Resultado esperado

A primeira linha gera nomes novos por brace expansion. Um glob, por padrão, depende do conteúdo existente do diretório.

Mapa de responsabilidade: Bash: ambos os mecanismos, em fases diferentes. `mkdir`: recebe o resultado final.

Erro ou cuidado comum: Não ensine `*` como curinga “do `mkdir`”. É um padrão do shell que normalmente é expandido antes da execução.

Base técnica: *GNU Bash: Shell Expansions* • *GNU Bash: Brace Expansion* • *GNU Bash: Filename Expansion*

Conexão com o próximo capítulo

No capítulo seguinte veremos um uso de globbing que faz sentido junto com `mkdir`: criar subdiretórios sob diretórios já existentes selecionados por padrão.

CAPÍTULO 29 — mkdir e globbing

Objetivo: Usar filename expansion somente em um cenário natural: selecionar diretórios existentes e criar algo dentro deles.

Fundamento

Imagine que já existam `projeto-api` e `projeto-web`, e você queira garantir um subdiretório `logs` em cada um. `mkdir -p projeto-\*/logs` pode ser útil: o Bash tenta expandir o padrão `projeto-\*` contra entradas existentes e produz pathnames como `projeto-api/logs` e `projeto-web/logs`.

Como funciona

Esse exemplo mostra a direção correta do globbing: ele seleciona prefixos que já existem; o mkdir cria um componente novo abaixo deles. Ainda assim, a linha depende do estado do diretório e das opções de globbing do shell.

Por que isso importa

Se nenhuma correspondência existir, Bash padrão normalmente deixa o padrão não expandido; com `nullglob`, ele pode removê-lo. Essas diferenças podem produzir resultados surpreendentes, portanto scripts críticos devem controlar explicitamente as opções e validar as entradas.

Exemplo prático

```
mkdir -p projeto-*/logs
```

Resultado esperado

Com `projeto-api` e `projeto-web` existentes, o Bash pode entregar `projeto-api/logs` e `projeto-web/logs` ao mkdir.

Mapa de responsabilidade: Bash: filename expansion. mkdir: cria os subdiretórios finais.

Erro ou cuidado comum: Evite globbing quando você pretende gerar nomes que ainda não existem. Para isso, brace expansion, múltiplos operandos ou loops são mais adequados.

Base técnica: GNU Bash: Filename Expansion • GNU Bash: Shell Expansions

Conexão com o próximo capítulo

Chegamos ao momento de juntar as peças e acompanhar o que o Bash faz com uma linha antes de executar mkdir.

CAPÍTULO 30 — o que o Bash faz antes de executar mkdir

Objetivo: Acompanhar, em ordem, parsing e expansões relevantes até chegar à busca e execução do programa.

Fundamento

Uma linha de shell não é enviada “como texto inteiro” ao programa. Primeiro, o Bash analisa a sintaxe e separa tokens, reconhecendo operadores, palavras, redirecionamentos e quoting. Depois, para palavras de um comando simples, realiza expansões em ordem definida.

Como funciona

Na sequência geral do Bash, brace expansion vem primeiro. Depois ocorrem tilde expansion, parameter/variable expansion, arithmetic expansion e command substitution (com regras de avaliação da esquerda para a direita nessa fase). Em seguida vêm word splitting e filename expansion. Quote removal ocorre por último. Somente depois o shell chega a uma lista final de palavras que pode ser usada como nome do comando e argumentos.

Por que isso importa

Então entra a busca do comando: se ``mkdir`` não contém ``/``, o Bash verifica funções, builtins e, não encontrando um builtin chamado `mkdir`, procura um executável nos diretórios de ``$PATH``. O programa é iniciado em um ambiente de execução separado e recebe os argumentos já transformados. Isso é a base para entender por que ``~``, ``$HOME``, ``{a,b}``, ``*`` e ``$(...)`` não são “recursos do `mkdir`”.

Exemplo prático

```
raiz="$HOME/lab"
mkdir -p "$raiz"/{src,docs}
```

Resultado esperado

O Bash expande ``$raiz``, preserva o valor como uma palavra por causa das aspas, realiza brace expansion e monta os pathnames finais; só então executa `mkdir`.

Mapa de responsabilidade: Bash: parsing, expansões, quote removal, command search. `mkdir`: execução com argv final.

Erro ou cuidado comum: A lista de expansões é uma ordem de processamento, não uma receita para usar todas em cada comando. Quoting pode suprimir etapas específicas em certas partes da palavra.

Base técnica: *GNU Bash: Shell Expansions* • *GNU Bash: Simple Command Expansion* • *GNU Bash: Command Search and Execution*

Conexão com o próximo capítulo

Com o processamento do shell consolidado, começaremos a estudar permissões do diretório recém-criado.

PARTE IV — PERMISSÕES E METADADOS

CAPÍTULO 31 — mkdir e permissões padrão

Objetivo: Introduzir os bits de permissão e a influência da umask na criação de diretórios.

Fundamento

Quando um diretório é criado sem `-m`, o utilitário solicita permissões de base que, no modelo tradicional, partem de `rwX` para usuário, grupo e outros, e a máscara de criação do processo (umask) remove bits. No Linux sem default ACL no diretório pai, a relação da chamada de sistema pode ser expressa como `mode & ~umask & 0777`.

Como funciona

Uma umask comum como `022` remove escrita de grupo e outros, resultando tipicamente em `755` para um diretório criado a partir de `777`. Já `027` tende a produzir `750`. A palavra “tende” lembra que ACLs padrão e outras políticas podem participar.

Por que isso importa

A umask pertence ao processo/shell e é herdada por processos filhos. O mkdir usa o ambiente de criação existente; ele não inventa uma política independente de umask.

Exemplo prático

```
umask
mkdir projeto
ls -ld projeto
```

Resultado esperado

O modo observado depende da umask e, quando aplicável, de ACLs padrão do diretório pai.

Mapa de responsabilidade: Shell/processo: umask. mkdir/libc/syscall: solicita modo. Kernel/filesystem: aplica regras de criação/ACL/permissões.

Erro ou cuidado comum: Não calcule permissões finais apenas subtraindo números decimais. Umask opera como máscara de bits, não como aritmética comum.

Base técnica: *Linux man-pages: mkdir(2)* • *Linux man-pages: umask(2)*

Conexão com o próximo capítulo

A opção `-m` permite solicitar um modo explícito para o diretório final.

CAPÍTULO 32 — mkdir -m com modo numérico

Objetivo: Definir permissões do diretório final usando notação octal.

Fundamento

A opção `-m` (`--mode`) pertence ao mkdir e aceita a mesma sintaxe de modo usada por `chmod`. Na forma numérica, cada dígito representa combinações de leitura (4), escrita (2) e execução (1) para usuário, grupo e outros.

Como funciona

`mkdir -m 700` privado concede `rwX` apenas ao proprietário. `750` dá `rwX` ao proprietário, `r-X` ao grupo e nenhuma permissão a outros. `755` adiciona `r-X` a grupo e outros. Em diretórios, porém, o significado operacional de `r/w/X` será estudado separadamente.

Por que isso importa

No GNU `mkdir`, `-m` afeta os diretórios explicitamente dados na linha de comando, não os pais criados automaticamente por `-p`. Essa diferença é fundamental e será detalhada no capítulo 35.

Exemplo prático

```
mkdir -m 700 privado
mkdir -m 750 equipe
mkdir -m 755 publico
```

Resultado esperado

Cada diretório final é criado com o modo solicitado, sujeito às regras de ACL e detalhes da implementação.

Mapa de responsabilidade: `mkdir`: opção `-m`. Kernel/filesystem: aplica criação e metadados efetivos.

Erro ou cuidado comum: Não diga que `-m 755` significa “permissão 755 para todos”: cada dígito se refere a uma classe diferente.

Base técnica: GNU Coreutils: *mkdir invocation* • POSIX: *mkdir utility*

Conexão com o próximo capítulo

A mesma política pode ser expressa de forma simbólica, muitas vezes mais legível.

CAPÍTULO 33 — `mkdir -m` com modo simbólico

Objetivo: Usar modos simbólicos e relacioná-los às classes `u`, `g`, `o` e `a`.

Fundamento

Modos simbólicos descrevem alterações ou definições com letras. `u`, `g` e `o` representam usuário, grupo e outros; `a` representa todos. Operadores como `=`, `+` e `-` definem, adicionam ou removem permissões no modo calculado.

Como funciona

No GNU `mkdir`, a interpretação de `-m` parte de uma base equivalente a `a=rxw` para as permissões comuns. Assim, uma expressão como `u=rxw,g=rx,o=` descreve de forma direta um modo equivalente a `750`.

Por que isso importa

A notação simbólica pode ser mais expressiva quando o objetivo didático é mostrar intenção. Porém, scripts que precisam de portabilidade devem observar a sintaxe especificada pelo POSIX para o argumento de modo.

Exemplo prático

```
mkdir -m u=rwx,g=rx,o= equipe
```

Resultado esperado

Cria `equipe` com `rwx` para o proprietário, `r-x` para o grupo e nenhuma permissão para outros.

Mapa de responsabilidade: `mkdir`: interpreta o argumento de `-m` segundo regras de modo.

Erro ou cuidado comum: Ao explicar modos, não trate r/w/x em diretórios como se fossem idênticos aos efeitos em arquivos regulares.

Base técnica: *GNU Coreutils: mkdir invocation* • *POSIX: mkdir utility*

Conexão com o próximo capítulo

O próximo capítulo é dedicado exatamente ao significado de r, w e x em diretórios.

CAPÍTULO 34 — r, w e x em diretórios

Objetivo: Compreender as permissões de diretório como controle sobre listagem, modificação de entradas e travessia/pesquisa.

Fundamento

Em um diretório, `r` permite ler a lista de nomes das entradas; `x` é a permissão de busca/travessia, necessária para acessar entradas pelo nome e atravessar aquele componente em um pathname; `w` permite modificar a lista de entradas, isto é, criar, remover ou renomear nomes dentro do diretório, normalmente em combinação com `x`.

Como funciona

Por isso, um diretório com leitura sem execução pode permitir obter nomes em certas operações, mas impedir acesso normal aos objetos por pathname. Escrita sem execução também não entrega a capacidade prática esperada de criar/remover entradas, porque a busca dos nomes fica bloqueada.

Por que isso importa

Esses efeitos mostram por que permissões de diretório são mais sobre o namespace do que sobre o conteúdo dos arquivos. O conteúdo de um arquivo regular é governado pelas permissões daquele arquivo; o diretório governa a capacidade de localizar e alterar entradas de nome.

Exemplo prático

```
mkdir -m 750 equipe
```

Resultado esperado

O modo `750` significa, no diretório: proprietário pode listar, modificar entradas e atravessar; grupo pode listar e atravessar; outros não têm acesso por esses bits.

Mapa de responsabilidade: `mkdir`: apenas define o modo solicitado. Kernel/VFS/filesystem: faz as verificações de acesso nas operações posteriores.

Erro ou cuidado comum: Não descreva ``w`` em diretório como “editar os arquivos dentro”. Ela controla entradas do diretório; a escrita no conteúdo do arquivo depende do próprio arquivo.

Base técnica: *Linux permission model* • *Linux man-pages: mkdir(2)*

Conexão com o próximo capítulo

Agora combinaremos ``-p`` e ``-m``, onde aparece uma diferença importante entre o diretório final e os pais criados automaticamente.

CAPÍTULO 35 — `mkdir -p` combinado com `-m`

Objetivo: Explicar por que ``-m`` não é simplesmente aplicado a todos os diretórios criados por ``-p`` no GNU `mkdir`.

Fundamento

No GNU `mkdir`, ``-m`` afeta os diretórios que aparecem como operandos finais da linha de comando. Diretórios pais criados automaticamente por ``-p`` usam outra regra e não recebem simplesmente o mesmo modo solicitado para o final.

Como funciona

Por exemplo, ``mkdir -p -m 700 projeto/privado`` pode criar ``projeto`` e ``privado``, mas o ``700`` é direcionado ao operando final ``privado``. O modo do pai ``projeto`` depende da lógica de ``-p`` e da `umask`. O manual GNU recomenda controlar permissões dos pais via `umask` quando isso for necessário, ou ajustar metadados depois com ferramentas apropriadas.

Por que isso importa

Essa separação existe porque o programa precisa criar componentes intermediários utilizáveis para continuar a travessia e, ao mesmo tempo, respeitar a política de criação. Tratar todos os níveis de forma idêntica poderia impedir a própria continuação da criação.

Exemplo prático

```
umask 027
mkdir -p -m 700 projeto/privado
ls -ld projeto projeto/privado
```

Resultado esperado

O diretório final deve refletir o modo solicitado; o pai criado por ``-p`` segue a regra de pais/`umask`, não recebe automaticamente ``700``.

Mapa de responsabilidade: `mkdir: -p`` e ``-m`` com regras distintas para pais e operandos finais. `umask:` estado do processo.

Erro ou cuidado comum: Não generalize o resultado numérico do pai sem considerar `umask`, ACL padrão e implementação.

Base técnica: *GNU Coreutils: mkdir invocation*

Conexão com o próximo capítulo

Além dos nove bits `rwX`, diretórios podem usar bits especiais como `setgid` e `sticky`.

CAPÍTULO 36 — bits especiais em diretórios

| **Objetivo:** Introduzir `setgid` e `sticky bit` em seus contextos práticos.

Fundamento

Quando o bit `setgid` está ativo em um diretório no Linux, novos objetos normalmente herdam o grupo do diretório pai; novos subdiretórios também recebem o bit `setgid`. Isso é útil em áreas compartilhadas por uma equipe, pois mantém a associação de grupo coerente.

Como funciona

O `sticky bit` em diretórios, famoso em `/tmp`, restringe a remoção e renomeação de entradas: em termos simplificados, usuários não privilegiados ficam limitados aos arquivos que possuem, ao diretório que possuem ou a outras condições autorizadas. Isso evita que qualquer usuário com escrita no diretório remova indiscriminadamente arquivos de outros.

Por que isso importa

GNU `mkdir -m` pode mencionar bits especiais como extensão, mas o tratamento exato e a janela temporária mencionada pelo manual GNU merecem cuidado. Para ensino inicial, é melhor entender primeiro a semântica do bit e depois observar como a ferramenta o solicita.

Exemplo prático

```
mkdir -m 2775 equipe-compartilhada
mkdir -m 1777 area-temporaria
```

Resultado esperado

Os modos numéricos incluem, respectivamente, `setgid` (`2`) e `sticky` (`1`) no dígito especial.

| **Mapa de responsabilidade:** `mkdir` GNU: pode solicitar bits especiais. Kernel/filesystem: implementa herança e restrições associadas.

| **Erro ou cuidado comum:** `Setuid` em diretórios não possui uma semântica universal útil comparável ao `setgid`; evite transferir automaticamente conceitos de executáveis para diretórios.

Base técnica: *Linux man-pages: inode(7)* • *GNU Coreutils: mkdir invocation*

Conexão com o próximo capítulo

No próximo capítulo veremos como proprietário e grupo do novo diretório são escolhidos.

CAPÍTULO 37 — proprietário e grupo do diretório

| **Objetivo:** Relacionar `effective UID`, grupo efetivo, diretório pai e herança `setgid`.

Fundamento

No Linux, um diretório recém-criado pertence ao `effective user ID` do processo criador. A escolha do grupo depende da política: normalmente usa o `effective group ID` do processo, mas se o diretório pai tiver `setgid` (ou o filesystem estiver em uma semântica de grupo correspondente), o novo diretório herda o grupo do pai.

Como funciona

Quando o pai tem `setgid`, um subdiretório novo também herda o próprio bit `setgid`. Essa combinação cria áreas colaborativas onde novos níveis mantêm o grupo da árvore.

Por que isso importa

O `mkdir` não é um comando de ``chown``. Ele pede a criação; o kernel e o filesystem determinam ownership dentro das regras e credenciais do processo. Se o objetivo for alterar proprietário/grupo depois, entram ferramentas e privilégios separados.

Exemplo prático

```
mkdir equipe/novo-modulo
```

Resultado esperado

O proprietário tende a ser o effective UID do processo; o grupo depende da política do pai e de `setgid`.

Mapa de responsabilidade: Processo/kernel/filesystem: ownership. `mkdir`: solicita a criação.

Erro ou cuidado comum: Não presuma que o grupo sempre será o grupo primário do usuário; `setgid` do pai e opções de montagem podem alterar a regra.

Base técnica: *Linux man-pages: mkdir(2)* • *Linux man-pages: inode(7)*

Conexão com o próximo capítulo

A última camada de permissões que estudaremos é a interação entre `umask` e ACL padrão.

CAPÍTULO 38 — `mkdir`, `umask` e ACL padrão

Objetivo: Entender por que uma default ACL no diretório pai altera o cálculo tradicional baseado em `umask`.

Fundamento

No Linux, uma default ACL associada ao diretório pai funciona como modelo de herança para novos objetos. Quando ela existe, o algoritmo de criação usa essa ACL padrão e limita as permissões herdadas de acordo com o modo solicitado. A `umask` tradicional deixa de ser aplicada do mesmo modo simples.

Como funciona

Sem default ACL, o modelo clássico usa o modo solicitado e a `umask` para determinar as permissões iniciais. Com default ACL, o novo objeto herda entradas da ACL do pai, e essas entradas são restringidas para não conceder permissões além das permitidas pelo modo de criação.

Por que isso importa

Isso explica por que duas máquinas com a mesma `umask` podem produzir resultados diferentes em diretórios pais com políticas ACL distintas. Diagnóstico de permissões precisa observar não apenas ``umask``, mas também ACLs do caminho.

Exemplo prático

```
getfacl .  
mkdir projeto  
getfacl projeto
```

Resultado esperado

Se houver default ACL no diretório pai, o novo diretório pode herdar entradas ACL; sem ela, prevalece o cálculo tradicional com umask.

Mapa de responsabilidade: ACL/kernel/filesystem: herança. umask: máscara tradicional quando não substituída pela default ACL. mkdir: solicita o modo de criação.

Erro ou cuidado comum: Não diga simplesmente “umask sempre define a permissão final”. Default ACL é uma exceção importante no Linux.

Base técnica: *Linux man-pages: acl(5)* • *Linux man-pages: mkdir(2)* • *Linux man-pages: umask(2)*

Conexão com o próximo capítulo

Com permissões estabelecidas, entraremos nos principais erros e em como interpretá-los sem decorar mensagens.

PARTE V — FALHAS E DIAGNÓSTICO

CAPÍTULO 39 — erro “File exists”

Objetivo: Entender a condição `EEXIST` e comparar o `mkdir` comum com o comportamento de `-p` quando o caminho final já é diretório.

Fundamento

Se o pathname final já existir, a criação comum não pode simplesmente substituir o objeto. A chamada de sistema reporta uma condição do tipo `EEXIST`; o GNU `mkdir` apresenta um diagnóstico como “File exists” dependendo do locale.

Como funciona

Importante: `EEXIST` não prova que o objeto existente seja diretório. Pode ser arquivo regular, link simbólico ou outro tipo. O `mkdir` comum falha porque já existe uma entrada naquele nome.

Por que isso importa

Com `-p`, o caso específico em que o operando já nomeia um diretório é tolerado sem erro. Isso não significa ignorar qualquer `EEXIST`; se houver um arquivo regular onde o caminho precisa ser diretório, a operação continua inviável.

Exemplo prático

```
mkdir projeto
mkdir projeto
mkdir -p projeto
```

Resultado esperado

A segunda linha comum falha se `projeto` já existe. A terceira é aceita se `projeto` for de fato um diretório.

Mapa de responsabilidade: Kernel: `EEXIST` quando a entrada já existe. `mkdir`: decide como tratar isso; `-p` tolera o caso diretório.

Erro ou cuidado comum: Não traduza todo “File exists” para “a pasta já existe”; o nome pode pertencer a outro tipo de objeto.

Base técnica: *Linux man-pages: `mkdir(2)`* • *GNU Coreutils: `mkdir invocation`*

Conexão com o próximo capítulo

O próximo capítulo mostra explicitamente por que `-p` não resolve um componente que é arquivo regular.

CAPÍTULO 40 — `mkdir -p` não resolve tudo

Objetivo: Demonstrar o conflito em que um componente do pathname não é diretório.

Fundamento

Considere um arquivo regular chamado `projeto`. Executar `mkdir -p projeto/src` exige usar `projeto` como componente intermediário de diretório. A resolução encontra uma entrada com esse nome, mas ela não é diretório; por isso o caminho não pode continuar.

Como funciona

Essa condição está associada a `ENOTDIR`: um componente usado como diretório não é, de fato, um diretório. `-p` não converte arquivos, não remove conflitos e não escolhe automaticamente outro caminho.

Por que isso importa

Automação robusta deve distinguir “diretório ausente” de “nome ocupado por objeto incompatível”. O primeiro caso é resolvido por `-p`; o segundo exige decisão explícita do operador/script.

Exemplo prático

```
printf 'x' > projeto
mkdir -p projeto/src
```

Resultado esperado

A criação falha porque `projeto` é arquivo regular e não pode ser atravessado como diretório.

Mapa de responsabilidade: `printf`/redirecionamento: preparam o cenário. `mkdir`: tenta garantir a árvore. Kernel/VFS: retorna `ENOTDIR` durante a resolução.

Erro ou cuidado comum: Nunca use `-p` como justificativa para omitir validações quando entradas podem colidir com arquivos ou links inesperados.

Base técnica: *Linux man-pages: mkdir(2)* • *Linux man-pages: path_resolution(7)*

Conexão com o próximo capítulo

Agora compararemos `ENOENT` e `ENOTDIR`, dois erros de resolução que indicam causas diferentes.

CAPÍTULO 41 — `ENOENT` e `ENOTDIR`

Objetivo: Distinguir componente ausente de componente existente com tipo inadequado.

Fundamento

`ENOENT` aparece quando um componente necessário não existe (ou há, por exemplo, um link simbólico pendente em uma posição relevante). `ENOTDIR` aparece quando um componente que precisa ser atravessado como diretório existe, mas não é diretório.

Como funciona

Essa distinção ajuda a diagnosticar sem depender apenas da frase traduzida. `mkdir pai/filho` com `pai` ausente aponta para `ENOENT`; com `pai` sendo arquivo regular, aponta para `ENOTDIR`. A linha digitada é parecida, mas o estado do filesystem é diferente.

Por que isso importa

No nível do utilitário, a mensagem exibida é derivada dessas condições reportadas pelas camadas inferiores. Em scripts, o status de saída continuará apenas indicando falha em termos gerais; para conhecer a causa, você precisa do diagnóstico e/ou de validações específicas.

Exemplo prático

```
mkdir ausente/filho
# contraste: um arquivo regular no lugar de "ausente" levaria a ENOTDIR
```

Resultado esperado

O primeiro caso falha porque o componente pai não existe; o segundo cenário conceitual falha porque o componente não é diretório.

Mapa de responsabilidade: Kernel/VFS: condições ENOENT/ENOTDIR. mkdir: converte a falha em diagnóstico/status.

Erro ou cuidado comum: Mensagens podem ser localizadas; nomes simbólicos de erro são conceitos de programação/sistema e não aparecem necessariamente no texto do terminal.

Base técnica: *Linux man-pages: mkdir(2)* • *Linux man-pages: path_resolution(7)*

Conexão com o próximo capítulo

Além de estrutura do pathname, a criação pode falhar por política de acesso ou estado do filesystem.

CAPÍTULO 42 — Permission denied, filesystem cheio e somente leitura

Objetivo: Relacionar EACCES, ENOSPC e EROFS a causas operacionais sem reduzir o capítulo a uma tabela de erro.

Fundamento

‘EACCES’ pode ocorrer quando o processo não tem permissão de escrita no diretório pai ou não tem permissão de busca em algum componente do caminho. Portanto, “Permission denied” não significa necessariamente falta de ‘w’ apenas no diretório final; o caminho inteiro participa da resolução.

Como funciona

‘ENOSPC’ indica falta de recursos para criar a nova entrada, como espaço/alocação disponível no filesystem; quotas também podem gerar condições relacionadas. ‘EROFS’ indica que o local relevante está em filesystem montado somente para leitura.

Por que isso importa

Esses casos mostram por que repetir o mesmo comando não corrige todas as falhas. O tratamento depende da causa: permissões exigem política/credenciais adequadas; falta de espaço exige liberar/provisionar recursos; read-only exige entender por que o filesystem está montado dessa forma.

Exemplo prático

```
mkdir /root/teste
mkdir /caminho-em-filesystem-somente-leitura/teste
```

Resultado esperado

Os resultados dependem das credenciais e mounts do sistema. O objetivo é interpretar a classe de falha, não reproduzir uma mensagem específica em toda máquina.

Mapa de responsabilidade: Kernel/filesystem: verifica permissões, espaço e flags de montagem. `mkdir`: reporta falha.

Erro ou cuidado comum: Não recomende ``sudo`` automaticamente para “resolver Permission denied”. Ele altera o contexto de privilégio e pode mascarar uma política ou pathname incorreto.

Base técnica: *Linux man-pages: mkdir(2)*

Conexão com o próximo capítulo

O último capítulo de erros mostra uma característica importante de múltiplos operandos: sucesso parcial.

CAPÍTULO 43 — vários operandos e sucesso parcial

Objetivo: Entender que uma invocação pode criar alguns diretórios e falhar em outros.

Fundamento

O `mkdir` processa operandos na ordem dada. Se ``mkdir a b c`` cria ``a`` e ``b``, mas encontra uma falha em ``c``, os dois primeiros não são automaticamente desfeitos. A execução não oferece uma transação de rollback para o conjunto.

Como funciona

Isso importa em scripts: status não zero da invocação indica que houve falha, mas não significa que nenhum efeito aconteceu. O estado parcial precisa ser considerado ao decidir se o script deve limpar, continuar, registrar ou tentar novamente.

Por que isso importa

O comportamento também reforça a utilidade de ``-p`` em tarefas de convergência estrutural: uma repetição pode reaproveitar partes já criadas. Mas, novamente, isso não resolve conflitos ou metadados incorretos por si só.

Exemplo prático

```
mkdir alpha beta /local/sem-permissao/gamma
```

Resultado esperado

Dependendo do ambiente, ``alpha`` e ``beta`` podem ser criados antes de a criação do terceiro operando falhar.

Mapa de responsabilidade: `mkdir`: processa operandos sequencialmente e retorna status global da invocação.

Erro ou cuidado comum: Não assuma atomicidade de lote. Se atomicidade lógica for necessária, o script precisa projetar validação/rollback explicitamente.

Base técnica: *GNU Coreutils: mkdir invocation* • *POSIX: mkdir utility*

Conexão com o próximo capítulo

Com fundamentos, pathnames, expansões, permissões e erros prontos, passaremos à automação com variáveis Bash.

PARTE VI — VARIÁVEIS, SCRIPTS E AUTOMAÇÃO

CAPÍTULO 44 — mkdir com variáveis Bash

Objetivo: Armazenar nomes/pathnames em variáveis e usar parameter expansion para formar argumentos do mkdir.

Fundamento

Variáveis permitem separar dado de estrutura de comando. ``diretorio=projeto`` armazena uma string no shell; ``mkdir "$diretorio"`` faz parameter expansion e entrega o valor como argumento ao mkdir.

Como funciona

Essa técnica é central em scripts porque o nome pode vir de configuração, parâmetro, cálculo ou entrada validada. O mkdir continua sem saber que houve uma variável: ele recebe apenas a string final.

Por que isso importa

O quoting ao redor da expansão merece destaque. Sem aspas, o resultado pode sofrer word splitting e filename expansion em contextos comuns. Com aspas duplas, o valor normalmente permanece um único argumento, mesmo se contiver espaços.

Exemplo prático

```
diretorio="projeto de teste"
mkdir "$diretorio"
```

Resultado esperado

É solicitado um único diretório chamado ``projeto de teste``.

Mapa de responsabilidade: Bash: variável e parameter expansion. mkdir: recebe um argumento final.

Erro ou cuidado comum: Variável não é “parâmetro do mkdir”. Ela existe na linguagem do shell e desaparece como conceito antes da execução do programa.

Base técnica: GNU Bash: Shell Parameter Expansion • GNU Bash: Word Splitting

Conexão com o próximo capítulo

O próximo capítulo aprofunda exatamente a diferença entre expansão citada e não citada.

CAPÍTULO 45 — variáveis e quoting

Objetivo: Entender por que `"$diretorio"` é a forma normalmente segura para um pathname armazenado em variável.

Fundamento

Se ``diretorio="meu projeto"``, a expansão não citada ``mkdir $diretorio`` produz texto que, em um contexto comum, passa por word splitting e pode virar dois argumentos: ``meu`` e ``projeto``. Depois, cada palavra também pode participar de filename expansion.

Como funciona

Com ``mkdir "$diretorio"``, as aspas duplas preservam o resultado da parameter expansion como uma única palavra (salvo casos especiais como arrays e ``"$@"``). O valor contém o espaço, mas o shell não o usa como separador de argumentos.

Por que isso importa

Essa regra é uma das bases de scripts de shell robustos: quote expansões quando o valor deve representar exatamente um pathname/argumento. Remover aspas deve ser uma decisão consciente porque você quer splitting/globbering, não um hábito.

Exemplo prático

```
diretorio="meu projeto"
mkdir "$diretorio"
```

Resultado esperado

O `mkdir` recebe um único operando com espaço.

Mapa de responsabilidade: Bash: parameter expansion + quoting. `mkdir`: argumento final.

Erro ou cuidado comum: Não confie em “meus nomes nunca têm espaço”. Caracteres especiais podem vir de entrada externa ou mudanças futuras.

Base técnica: *GNU Bash: Shell Parameter Expansion* • *GNU Bash: Word Splitting* • *GNU Bash: Filename Expansion*

Conexão com o próximo capítulo

A parameter expansion também oferece valores padrão e outras transformações úteis antes do `mkdir`.

CAPÍTULO 46 — parameter expansion

Objetivo: Usar formas como ``${nome:-projeto}`` para definir valores padrão sem atribuir essa lógica ao `mkdir`.

Fundamento

``${nome:-projeto}`` significa: se ``nome`` estiver unset ou vazio, produza ``projeto``; caso contrário, produza o valor de ``nome``. Essa decisão é tomada pelo Bash durante parameter expansion.

Como funciona

Com ``mkdir "${nome:-projeto}"``, o `mkdir` recebe apenas um argumento. Ele não conhece a variável, o operador ``:-`` ou a ideia de “valor padrão”. Essa separação permite usar recursos avançados do shell sem confundir a interface do utilitário.

Por que isso importa

Parameter expansion possui muitas formas, mas é melhor introduzi-las conforme uma necessidade real. Para `mkdir`, defaults, remoção de prefixos/sufixos e validação podem ser úteis; transformações excessivas tendem a esconder a intenção do script.

Exemplo prático

```
nome=""
mkdir "${nome:-projeto}"
```

Resultado esperado

Como `nome` está vazio, o Bash produz `projeto` e o mkdir tenta criar esse diretório.

Mapa de responsabilidade: Bash: `\${...}`. mkdir: criação do valor resultante.

Erro ou cuidado comum: `:-` não é operador do mkdir. Também não deve ser confundido com operador lógico do shell.

Base técnica: GNU Bash: Shell Parameter Expansion

Conexão com o próximo capítulo

Outra forma de construir dinamicamente um nome é usar a saída de outro comando por command substitution.

CAPÍTULO 47 — command substitution

Objetivo: Inserir a saída de um comando em parte de um nome usando `\${...}` e preservar a fronteira entre comandos.

Fundamento

Command substitution executa um comando e substitui a construção `\${...}` por sua saída padrão, removendo quebras de linha finais. Em `mkdir "backup-\$(date +%Y-%m-%d)"`, o Bash executa `date`, obtém o texto e monta um único argumento para mkdir.

Como funciona

Existem três responsabilidades: `date` produz um valor; Bash realiza command substitution e quoting; mkdir cria o pathname resultante. Essa linha é compacta, mas continua sendo uma composição de componentes independentes.

Por que isso importa

Em scripts, command substitution deve ser usada quando o valor realmente precisa vir de um comando. Para dados que já estão em variáveis, uma expansão direta é mais simples e evita processos extras.

Exemplo prático

```
mkdir "backup-$(date +%Y-%m-%d)"
```

Resultado esperado

Em uma data hipotética 2026-09-25, o argumento poderia se tornar `backup-2026-09-25`.

Mapa de responsabilidade: date: programa que produz texto. Bash: command substitution. mkdir: criação.

Erro ou cuidado comum: A saída substituída ainda precisa ser tratada com quoting apropriado. E resultados dinâmicos podem colidir com nomes já existentes.

Base técnica: *GNU Bash: Command Substitution*

Conexão com o próximo capítulo

Loops entram quando precisamos repetir `mkdir` com lógica por item, não apenas gerar uma lista de operandos.

CAPÍTULO 48 — `mkdir` em loops Bash

Objetivo: Escolher loops quando cada iteração precisa de lógica, validação ou tratamento individual.

Fundamento

Múltiplos operandos e brace expansion são ideais quando você já conhece a lista de nomes. Um loop é útil quando cada item exige decisão própria, logging, transformação ou reação a falhas.

Como funciona

Em um `for`, o Bash controla as iterações e executa o `mkdir` a cada vez. Isso significa múltiplas invocações, cada uma com seu próprio status. Você pode, por exemplo, continuar após uma falha, parar imediatamente ou registrar qual item falhou.

Por que isso importa

Loops não tornam o `mkdir` mais poderoso; eles tornam a linguagem ao redor dele mais expressiva. A escolha entre uma invocação com vários operandos e várias invocações em loop deve considerar clareza e necessidade de controle.

Exemplo prático

```
for diretorio in src docs tests; do
    mkdir "$diretorio" || printf 'falhou: %s\n' "$diretorio" >&2
done
```

Resultado esperado

O Bash executa o `mkdir` uma vez por item e pode reagir individualmente ao status de cada execução.

Mapa de responsabilidade: Bash: `for`, variável, `|`, `|>`, redirecionamento. `mkdir`: criação por iteração.

Erro ou cuidado comum: Não use loop apenas para imitar `mkdir a b c` quando nenhuma lógica por item é necessária.

Base técnica: *GNU Bash: Looping Constructs* • *GNU Bash: Lists of Commands*

Conexão com o próximo capítulo

O status do `mkdir` também pode ser usado diretamente como condição de um `if`.

CAPÍTULO 49 — `mkdir` em condicionais

Objetivo: Usar o próprio status do `mkdir` como condição de um `if`, sem testes redundantes de ``$?``.

Fundamento

No Bash, a condição de um `if` é uma lista de comandos; o ramo `then` é executado quando o status dessa lista é zero. Portanto, `if mkdir projeto; then ... fi` usa diretamente o resultado real da criação.

Como funciona

Não é necessário escrever `mkdir projeto; if [ $? -eq 0 ]; then ...`. Essa forma é mais frágil e verbosa, pois depende de capturar corretamente o último status. Colocar o comando diretamente na condição preserva a relação causal.

Por que isso importa

Essa técnica é útil quando o tratamento de sucesso/falha exige mais de um comando e o simples `&&`'|`` ficaria difícil de ler.

Exemplo prático

```
if mkdir projeto; then
    printf 'diretório criado
'
else
    printf 'não foi possível criar
' >&2
fi
```

Resultado esperado

O Bash escolhe o ramo com base no status retornado pelo `mkdir`.

Mapa de responsabilidade: `mkdir`: condição real. Bash: estrutura `if`. `printf`: mensagens de cada ramo.

Erro ou cuidado comum: Em scripts, mensagem de erro própria não deve apagar o diagnóstico útil do `mkdir`, a menos que você trate a causa conscientemente.

Base técnica: *GNU Bash: Conditional Constructs* • *GNU Bash: Exit Status*

Conexão com o próximo capítulo

No próximo capítulo, o `pathname` virá de um argumento passado ao script.

CAPÍTULO 50 — script recebendo diretório como argumento

Objetivo: Introduzir parâmetros posicionais, validação mínima e quoting ao criar um diretório fornecido pelo usuário.

Fundamento

Um script recebe argumentos como parâmetros posicionais. O primeiro é `$1`. Como ele pode estar ausente ou conter espaços, o script deve validar e citar a expansão antes de usá-la como `pathname`.

Como funciona

Em Bash, `[[ -z $destino ]]` é uma construção condicional da linguagem. Depois da validação, `mkdir -p -- "$destino"` usa `-p` para garantir a estrutura e `--` para proteger nomes cujo primeiro componente relevante possa começar com hífen.

Por que isso importa

Esse exemplo já mostra uma pequena interface de automação: o script aceita um dado externo, verifica uma pré-condição e entrega um argumento bem formado ao `mkdir`.

Exemplo prático

```
#!/usr/bin/env bash

destino=${1-}
if [[ -z $destino ]]; then
    printf 'uso: %s DIRETORIO\n' "$0" >&2
    exit 2
fi

mkdir -p -- "$destino"
```

Resultado esperado

Com um argumento válido, o script tenta garantir a existência do diretório solicitado.

Mapa de responsabilidade: Bash: parâmetros posicionais, `[[ ]]`, expansão, quoting. `mkdir`: `-p` e criação.

Erro ou cuidado comum: Se o script for POSIX sh em vez de Bash, `[[ ]]` não é portátil; use as construções especificadas para o shell-alvo.

Base técnica: *GNU Bash: Positional Parameters* • *GNU Bash: Conditional Constructs* • *GNU Coreutils: Common options*

Conexão com o próximo capítulo

Com essas peças, podemos montar um pequeno projeto real de automação estruturada.

CAPÍTULO 51 — projeto prático de automação

Objetivo: Construir um script pequeno que cria uma estrutura de projeto com tratamento de falha e mensagem clara.

Fundamento

O objetivo é reunir recursos já estudados sem transformar o script em demonstração de truques. O nome da raiz vem do primeiro argumento ou recebe um padrão; brace expansion gera os ramos; `-p` garante os pais; quoting protege o prefixo; e o status do `mkdir` decide se o script continua.

Como funciona

Observe que o `mkdir` continua protagonizando a alteração de estado. O ``printf`` apenas informa o resultado. Em um projeto maior, seria possível adicionar permissões, logs e validações, mas cada responsabilidade deveria permanecer explícita.

Por que isso importa

Esse tipo de script é útil em aulas porque conecta terminal interativo a automação: a mesma operação básica de criação se torna um bloco reutilizável dentro de uma ferramenta.

Exemplo prático

```
#!/usr/bin/env bash
set -u

raiz=${1:-projeto}

if mkdir -p -- "$raiz"/{src,docs,tests,frontend,backend}; then
    printf 'estrutura criada em: %s'
    ' "$raiz"
else
    printf 'falha ao criar a estrutura: %s'
    ' "$raiz" >&2
    exit 1
fi
```

Resultado esperado

O Bash gera cinco pathnames sob a raiz e uma única invocação do `mkdir` tenta garantir todos eles.

Mapa de responsabilidade: Bash: parâmetro padrão, brace expansion, ``if``, quoting. `mkdir`: ``-p`` e múltiplos operandos.

Erro ou cuidado comum: ``set -u`` foi incluído apenas como exemplo controlado de disciplina de script; outras opções como ``set -e`` exigem estudo próprio e não devem ser adicionadas automaticamente.

Base técnica: *GNU Bash Reference Manual* • *GNU Coreutils Manual*

Conexão com o próximo capítulo

Antes de entrar no kernel, precisamos organizar a origem de tudo que usamos: POSIX, GNU Coreutils e Bash.

PARTE VII — PORTABILIDADE E FUNCIONAMENTO INTERNO

CAPÍTULO 52 — POSIX, GNU Coreutils e Bash

Objetivo: Construir uma síntese de portabilidade e identificar a camada de origem de cada recurso.

Fundamento

POSIX especifica o utilitário `mkdir` com os operandos e opções fundamentais `-p` e `-m`, além de requisitos de comportamento. GNU Coreutils fornece a implementação comum em Ubuntu e adiciona recursos como `-v` e `-Z`, além de opções comuns GNU como `--help`, `--version` e o delimitador `--`.

Como funciona

Bash fornece a linguagem que cerca o programa: `;`, `&&`, `|`, `|`, variáveis, `$?`, tilde expansion, brace expansion, parameter expansion, command substitution, word splitting, filename expansion, loops e condicionais. Alguns desses recursos têm equivalentes ou subconjuntos em shells POSIX; brace expansion, em particular, é uma extensão do Bash e não deve ser atribuída ao POSIX `mkdir`.

Por que isso importa

Uma linha pode combinar três camadas: sintaxe do shell, opções do utilitário e comportamento do sistema operacional. Marcar essas fronteiras torna a portabilidade muito mais fácil de avaliar.

Exemplo prático

```
mkdir -p projeto          # -p: POSIX mkdir
mkdir -v projeto2         # -v: GNU mkdir
mkdir {a,b}               # chaves: Bash
```

Resultado esperado

Cada linha depende de uma camada diferente além da operação comum de criação.

Mapa de responsabilidade: POSIX: contrato portátil. GNU: implementação/extensões. Bash: linguagem e expansões. Linux: chamadas de sistema e filesystem.

Erro ou cuidado comum: Não use “Linux command” como rótulo que apaga as diferenças entre shell, Coreutils e interfaces do kernel.

Base técnica: *POSIX: mkdir utility* • *GNU Coreutils: mkdir invocation* • *GNU Bash Reference Manual*

Conexão com o próximo capítulo

Uma extensão GNU importante em sistemas com rotulagem de segurança é `-Z` para contexto SELinux.

CAPÍTULO 53 — `mkdir -Z` e SELinux

Objetivo: Entender a opção GNU de contexto de segurança sem pressupor SELinux ativo em todo Ubuntu.

Fundamento

GNU `mkdir` oferece `-Z/--context` para integração com contextos de segurança como SELinux. Sem contexto explícito, a opção pode ajustar o tipo de contexto segundo a política padrão do sistema, de modo semelhante à restauração de contexto; com contexto especificado, solicita um rótulo para os objetos recém-criados conforme as capacidades do sistema.

Como funciona

Esse recurso não é POSIX e seu efeito depende de mecanismos de segurança estarem disponíveis e habilitados. Não se deve concluir que todo Ubuntu usa SELinux ativamente só porque o binário GNU oferece a opção.

Por que isso importa

Em ambientes com SELinux, permissões tradicionais `rwX` não são toda a história: a política de segurança pode permitir ou negar operações adicionalmente. Por isso `-Z` pertence a um capítulo avançado e contextual.

Exemplo prático

```
mkdir -Z projeto
```

Resultado esperado

Em um sistema com suporte/política apropriados, o contexto pode ser ajustado conforme a configuração de segurança. Em outros ambientes, o comportamento pode incluir ausência de efeito útil ou aviso, conforme a forma usada.

Mapa de responsabilidade: GNU `mkdir`: opção `-Z`. SELinux/SMACK e política do sistema: interpretação/rotulagem efetiva.

Erro ou cuidado comum: Não apresente `-Z` como requisito para criar diretórios normalmente, nem como recurso universal de todos os sistemas POSIX.

Base técnica: GNU Coreutils: `mkdir invocation`

Conexão com o próximo capítulo

Chegamos ao caminho interno: o que acontece entre pressionar Enter e a solicitação chegar ao kernel.

CAPÍTULO 54 — do terminal ao kernel

Objetivo: Seguir as camadas terminal → shell → expansões → programa `mkdir` → biblioteca/interface de sistema → `mkdir()`/`mkdirat()` → kernel.

Fundamento

O terminal fornece a interface de entrada/saída da sessão; ele não interpreta semanticamente `mkdir`. O shell recebe a linha, analisa sua sintaxe, executa expansões, remove quoting e resolve qual comando deve ser executado. Para um `mkdir` externo encontrado em `$PATH`, o Bash inicia o programa e passa a lista final de argumentos.

Como funciona

O programa GNU `mkdir` valida opções, organiza os operandos e usa interfaces de biblioteca/sistema para solicitar a criação. Em sistemas Unix-like, essa solicitação chega a operações como ``mkdir()`` ou ``mkdirat()`` através da `libc` e da fronteira de `system call` apropriada. Detalhes exatos de wrappers podem variar entre versões e implementações; por isso é incorreto dizer que “digitar `mkdir` chama diretamente a `syscall mkdir`”.

Por que isso importa

No kernel, credenciais, resolução de `pathname`, permissões, mounts e filesystem entram no caminho. A chamada retorna sucesso ou erro à camada de usuário; o programa converte isso em diagnóstico/status; o shell recolhe o status e decide o que fazer a seguir.

Exemplo prático

```
mkdir laboratorio
# experimento opcional em ambiente de estudo:
strace -e trace=mkdir,mkdirat mkdir laboratorio2
```

Resultado esperado

``strace`` pode mostrar a fronteira de chamadas de sistema usada naquela implementação/versão, mas o resultado exato não deve ser universalizado.

Mapa de responsabilidade: Terminal: I/O. Bash: linguagem e execução. GNU `mkdir`: utilitário. `libc/syscall`: interface. Kernel: operação de sistema.

Erro ou cuidado comum: ``strace`` é ferramenta de observação, não parte do `mkdir`. E o wrapper usado pode variar; use a captura como evidência daquele ambiente.

Base técnica: *GNU Bash: Command Search and Execution* • *Linux man-pages: mkdir(2)*

Conexão com o próximo capítulo

O último capítulo abre a caixa do kernel um pouco mais: `pathname`, VFS, `dentries`, `inodes` e filesystem concreto.

CAPÍTULO 55 — `pathname`, VFS e filesystem

Objetivo: Construir uma visão tecnicamente sólida da criação sem presumir que todos os filesystems implementam internamente da mesma forma.

Fundamento

A resolução começa escolhendo o ponto inicial: raiz para `pathnames` absolutos, diretório de trabalho (ou um diretório referenciado por descritor em variantes ``*at``) para `pathnames` relativos. O kernel percorre os componentes, verificando permissões de busca e resolvendo entradas até chegar ao componente final que deve ser criado.

Como funciona

O VFS (Virtual File System) fornece uma camada comum para operações sobre diferentes filesystems. Durante lookup e criação, objetos em memória como `dentries` representam associações de nomes e ajudam no cache de resolução; `inodes` representam objetos do filesystem

e seus metadados/operações. A documentação do kernel descreve operações de inode específicas, incluindo uma operação ``mkdir`` implementada pelo filesystem.

Por que isso importa

Quando a criação é autorizada, o VFS encaminha a operação para a implementação concreta do filesystem. Ext4, XFS, tmpfs, NFS e outros não precisam organizar internamente seus metadados da mesma maneira. O resultado conceitual, porém, é que uma nova entrada de nome passa a referenciar um novo objeto diretório dentro das regras daquele filesystem.

Esse ponto final fecha o percurso iniciado no capítulo 1: ``mkdir pasta`` é uma linha curta, mas envolve uma cadeia de parsing, expansão, execução de processo, resolução de pathname, controles de acesso, VFS e implementação de filesystem. Saber usar o comando é o início; saber separar essas camadas é o que permite diagnosticar, automatizar e ensinar com precisão.

Exemplo prático

```
mkdir projeto
```

Resultado esperado

O mesmo comando simples do início agora pode ser explicado em todas as camadas, do shell à criação do objeto no filesystem.

Mapa de responsabilidade: Bash/utilitário/libc/kernel/VFS/filesystem: cada camada tem responsabilidade distinta.

Erro ou cuidado comum: Não diga que “o inode é o nome do arquivo” nem que dentries são necessariamente persistidas em disco. No VFS Linux, dentries são objetos em memória de cache/namespaces; a representação persistente é responsabilidade do filesystem.

Base técnica: *Linux man-pages: path_resolution(7)* • *Linux Kernel docs: VFS* • *Linux man-pages: mkdir(2)*

Conexão com o próximo capítulo

Fim da progressão principal. As seções finais consolidam opções, operadores, glossário e referências para revisão e uso como guia da playlist.

Resumo geral

O comando `mkdir` parece simples porque sua interface básica é pequena: receber um ou mais pathnames e criar diretórios. A complexidade real aparece quando observamos o ambiente ao redor. O Bash pode transformar a linha com quoting e expansões; o GNU `mkdir` acrescenta opções além do POSIX; permissões e ACLs influenciam os metadados iniciais; o kernel precisa resolver o pathname e autorizar a operação; o VFS encaminha a criação ao filesystem concreto.

A progressão deste livro foi desenhada para que cada nova camada responda a uma pergunta criada pela anterior. Primeiro: “como criar?”. Depois: “onde o nome é resolvido?”, “quem interpreta operadores e expansões?”, “quais permissões nascem com o diretório?”, “por que falhou?”, “como automatizar?” e, finalmente, “como a solicitação atravessa user space e kernel?”.

Ideia central: Aprender `mkdir` profundamente não é memorizar opções. É conseguir explicar, para qualquer linha de exemplo, quem interpreta cada símbolo, quais argumentos chegam ao programa, qual estado do sistema é necessário e em que camada um erro pode surgir.

Tabela — `mkdir` × Bash × outros comandos × sistema

Conceito	<code>mkdir</code>	Bash	Outro programa	Linux/filesystem
Criar diretório	<code>mkdir</code>	—	—	VFS/filesystem executam a criação
<code>`-p`, `-m`</code>	<code>mkdir</code> (POSIX)	—	—	Sistema aplica operações/permissões
<code>`-v`, `-Z`</code>	GNU <code>mkdir</code>	—	—	<code>`-Z`</code> depende de mecanismos de segurança
<code>`;`, `&&`, ` `</code>	—	Operadores/listas	—	—
<code>`~`, `\$HOME`</code>	—	Tilde/parameter expansion	—	—
<code>`{a,b}`</code>	—	Brace expansion	—	—
<code>`*`, `?`</code>	—	Filename expansion (globbing)	—	Filesystem fornece nomes existentes para lookup
<code>`cd`, `pwd`</code>	—	Builtins do shell	—	Kernel mantém cwd do processo
<code>`ls`, `mv`, `find`, `chmod`</code>	—	Shell apenas executa	Programas separados	Chamadas de sistema conforme cada operação
<code>umask</code>	—	Builtin/estado do processo	—	Afeta criação sem default ACL
ACL padrão	—	—	<code>getfacl/setfacl</code> são ferramentas	Kernel/filesystem aplicam herança
<code>mkdir()/mkdirat()</code>	—	—	libc expõe wrappers	Kernel recebe a operação

Tabela — opções e formas estudadas

Forma	Camada	Portabilidade	Função
<code>-p, --parents</code>	<code>mkdir</code>	POSIX + GNU	Cria pais ausentes e tolera operandos que já são diretórios.

-m MODE, --mode=MODE	mkdir	POSIX (`-m`) + GNU long option	Define o modo do diretório final criado; pais de `-p` seguem regra própria.
-v, --verbose	mkdir	GNU	Imprime mensagem para cada diretório criado.
-Z, --context[=CONTEXT]	mkdir	GNU	Integra contexto de segurança conforme suporte/política.
--	parser de opções do utilitário	Convenção GNU comum	Encerra opções; argumentos seguintes são operandos.
--help / --version	opções comuns GNU	GNU	Ajuda e versão; não alteram a criação.

Tabela — operadores e recursos de shell utilizados

Recurso	Origem	Papel
;	Bash/POSIX shell	Executa sequencialmente, sem condicionar o segundo ao sucesso do primeiro.
&&	Bash/POSIX shell	Executa a direita se a esquerda terminar com status zero.
	Bash/POSIX shell	Executa a direita se a esquerda terminar com status não zero.
\$?	parâmetro especial do shell	Expõe o status do comando mais recente.
~	tilde expansion (Bash)	Pode virar o diretório home antes da execução.
\$HOME / \${...}	parameter expansion	Substitui parâmetros/variáveis antes da execução.
{a,b}, {01..10}	brace expansion (Bash)	Gera palavras textualmente; não é POSIX mkdir.
\$(...)	command substitution	Insere a saída de um comando.
* ? []	filename expansion	Expandem padrões contra nomes existentes.
aspas	quoting	Controlam o significado especial e expansões/splitting.

Glossário

Argumento — Valor entregue a um comando/programa na invocação.

Operando — Argumento que representa o objeto principal sobre o qual o utilitário opera, como um pathname para mkdir.

Opção — Argumento que modifica o comportamento do utilitário, normalmente iniciado por ` ` ou `--`.

Pathname — String usada para identificar um objeto por uma sequência de componentes separados por `/`.

Componente — Parte de um pathname entre separadores `/`.

Caminho relativo — Pathname que não começa com `/`; normalmente parte do diretório de trabalho atual.

Caminho absoluto — Pathname que começa com `/`; parte da raiz do processo.

CWD — Current working directory: diretório de trabalho atual do processo.

Shell — Programa que interpreta a linguagem de comandos, executa expansões e inicia/utiliza comandos.

Builtin — Comando implementado dentro do próprio shell.

Expansão — Transformação de palavras pelo shell antes da execução, como parameter expansion ou globbing.

Quoting — Mecanismos que controlam o significado especial de caracteres e expansões.

Word splitting — Etapa em que certos resultados de expansão não citados podem ser divididos em palavras.

Globbing — Nome comum para filename expansion com padrões `*`, `?`` e classes.

Brace expansion — Geração textual de alternativas/sequências pelo Bash antes de outras expansões.

Status de saída — Valor numérico retornado por um comando/processo; zero representa sucesso para o shell.

errno — Código de erro usado por interfaces de sistema/biblioteca para descrever uma falha específica.

umask — Máscara de criação do processo que remove bits de permissão quando não substituída por default ACL.

ACL — Lista de controle de acesso que pode expressar permissões além do trio tradicional u/g/o.

Default ACL — ACL de diretório usada como modelo de herança para novos objetos criados nele.

setgid em diretório — Bit especial que influencia herança de grupo e é herdado por novos subdiretórios no Linux.

Sticky bit — Bit que restringe remoção/renomeação de entradas em diretórios graváveis compartilhados.

UID/GID efetivo — Credenciais efetivas do processo usadas em diversas decisões de acesso/ownership.

VFS — Camada Virtual File System do kernel Linux que oferece interface comum a diferentes filesystems.

Dentry — Objeto em memória do VFS associado a nomes e lookup no namespace.

Inode — Objeto que representa um objeto de filesystem e seus metadados/operações no VFS.

Filesystem — Implementação que organiza e persiste (ou simula, em pseudo filesystems) objetos e metadados.

Idempotência — Propriedade de uma operação cujo estado relevante não muda após repetições além da primeira aplicação bem-sucedida.

POSIX — Conjunto de padrões que especifica interfaces e utilitários portáveis em sistemas compatíveis.

GNU Coreutils — Coleção GNU que fornece utilitários básicos, incluindo a implementação de ``mkdir`` comum no Ubuntu.

SELinux — Mecanismo de segurança com rotulagem e políticas obrigatórias; pode interagir com ``mkdir -Z`` em sistemas configurados.

Referências técnicas

- **GNU Coreutils Manual — `mkdir` invocation**
https://www.gnu.org/software/coreutils/manual/html_node/mkdir-invocation.html
- **GNU Coreutils Manual — Common options**
https://www.gnu.org/software/coreutils/manual/html_node/Common-options.html
- **GNU Bash Reference Manual — Shell Expansions**
https://www.gnu.org/software/bash/manual/html_node/Shell-Expansions.html
- **GNU Bash Reference Manual — Lists of Commands**
https://www.gnu.org/software/bash/manual/html_node/Lists.html
- **GNU Bash Reference Manual — Command Search and Execution**
https://www.gnu.org/software/bash/manual/html_node/Command-Search-and-Execution.html
- **GNU Bash Reference Manual — Bourne Shell Builtins**
https://www.gnu.org/software/bash/manual/html_node/Bourne-Shell-Builtins.html
- **POSIX.1-2024 — `mkdir` utility**
<https://pubs.opengroup.org/onlinepubs/9799919799.2024edition/utilities/mkdir.html>
- **Linux man-pages — `mkdir(2)`**
<https://man7.org/linux/man-pages/man2/mkdir.2.html>
- **Linux man-pages — `path_resolution(7)`**
https://man7.org/linux/man-pages/man7/path_resolution.7.html
- **Linux man-pages — `pathname(7)`**
<https://man7.org/linux/man-pages/man7/pathname.7.html>
- **Linux man-pages — `umask(2)`**
<https://man7.org/linux/man-pages/man2/umask.2.html>
- **Linux man-pages — `acl(5)`**
<https://man7.org/linux/man-pages/man5/acl.5.html>
- **Linux man-pages — `inode(7)`**
<https://man7.org/linux/man-pages/man7/inode.7.html>
- **Linux Kernel Documentation — Overview of the Linux Virtual File System**
<https://docs.kernel.org/filesystems/vfs.html>

Conclusão

Ao concluir este percurso, o leitor deve ser capaz de olhar para uma linha simples como ``mkdir pasta`` e responder mais do que “ela cria uma pasta”. Deve saber que o Bash interpreta a linha e entrega argumentos a um programa; que pathnames relativos e absolutos têm pontos de partida diferentes; que opções como ``-p`` e ``-m`` pertencem ao utilitário, enquanto operadores e expansões pertencem ao shell; que permissões, umask, ACLs e bits especiais influenciam o novo objeto; que falhas expressam condições concretas da resolução e do filesystem; e que a operação finalmente atravessa a interface de sistema, VFS e implementação concreta do filesystem.

Este e-book acompanha a playlist do projeto Aprenda Linux BR e foi estruturado para funcionar como mapa de estudo, referência técnica e base de preparação das aulas. A recomendação é avançar na ordem: executar os exemplos, provocar os erros com segurança em um ambiente de laboratório e só então seguir para o próximo capítulo.

| **Aprenda Linux BR:** O objetivo é aprender → compreender → praticar → organizar → ensinar.